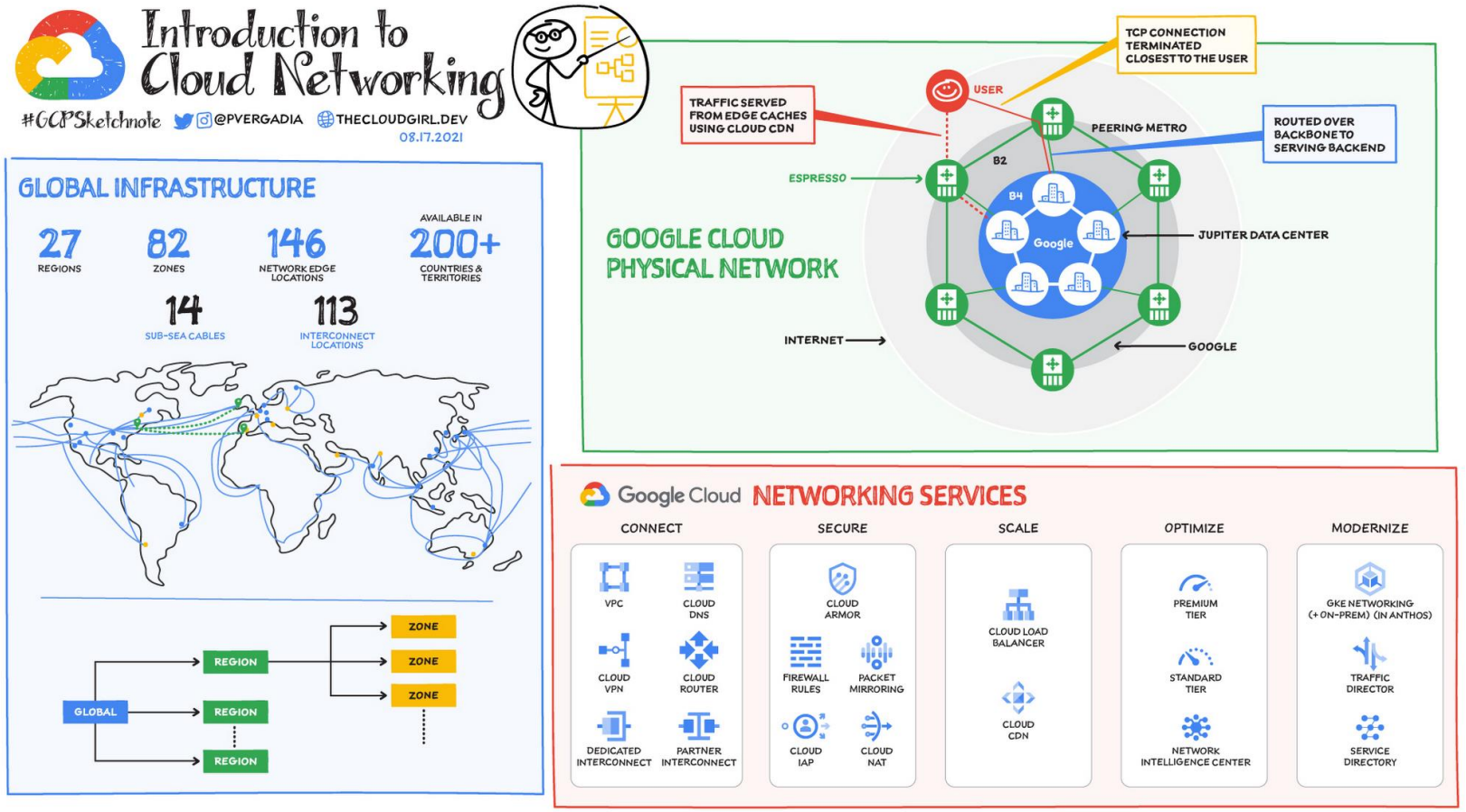
Network Services Overview

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



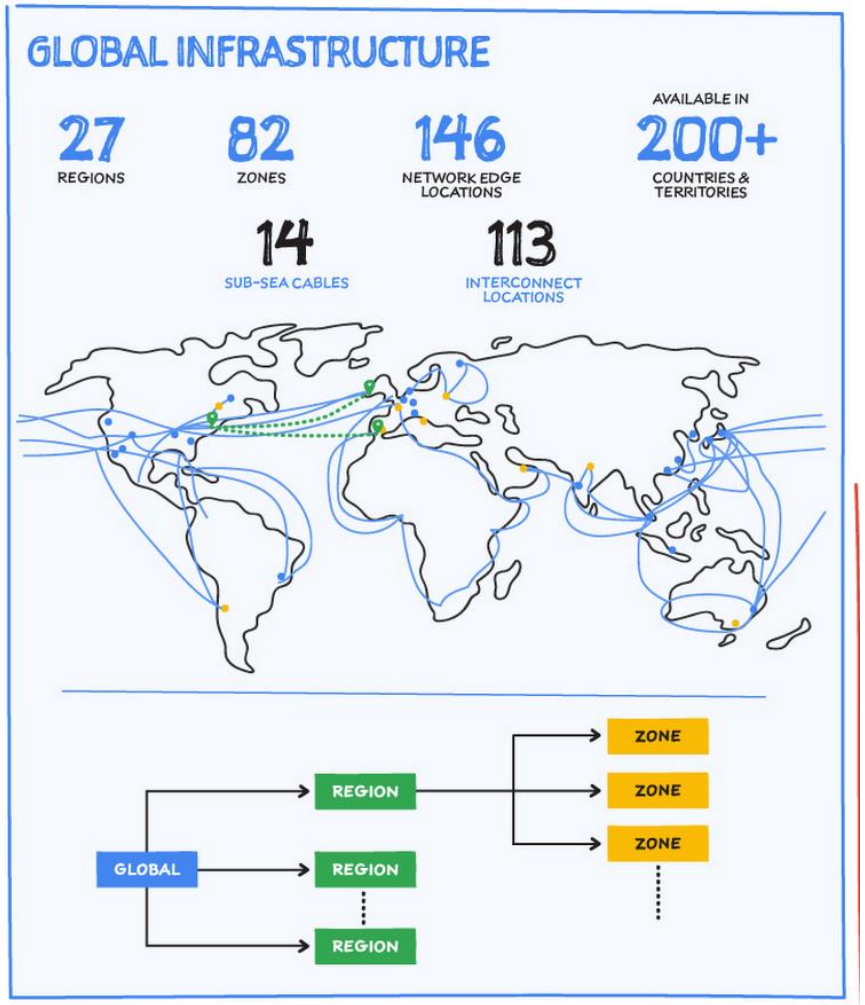
University of Colorado **Boulder**

Google Networking Overview



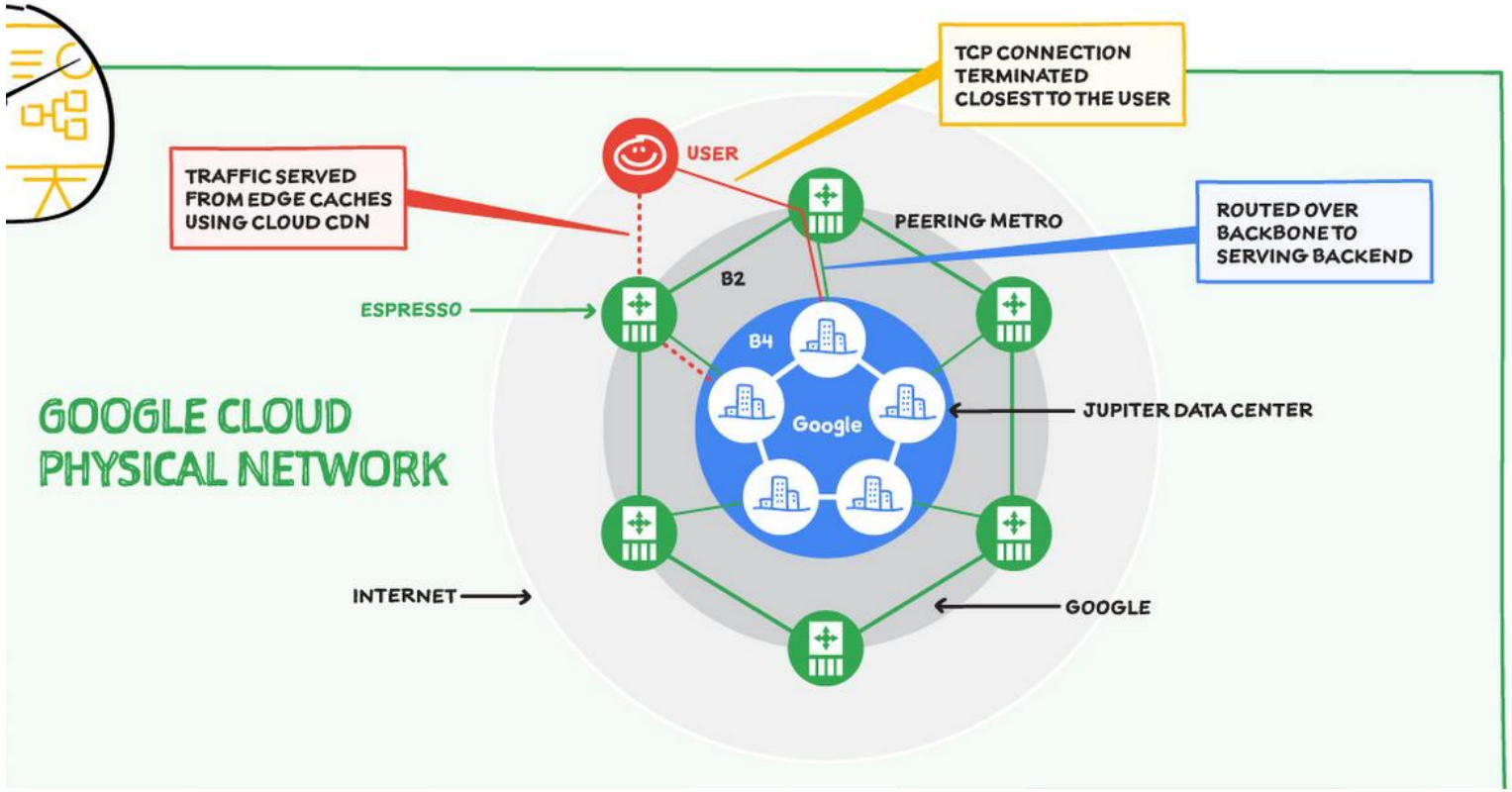
https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-networking-overview

How the Cloud is Organized



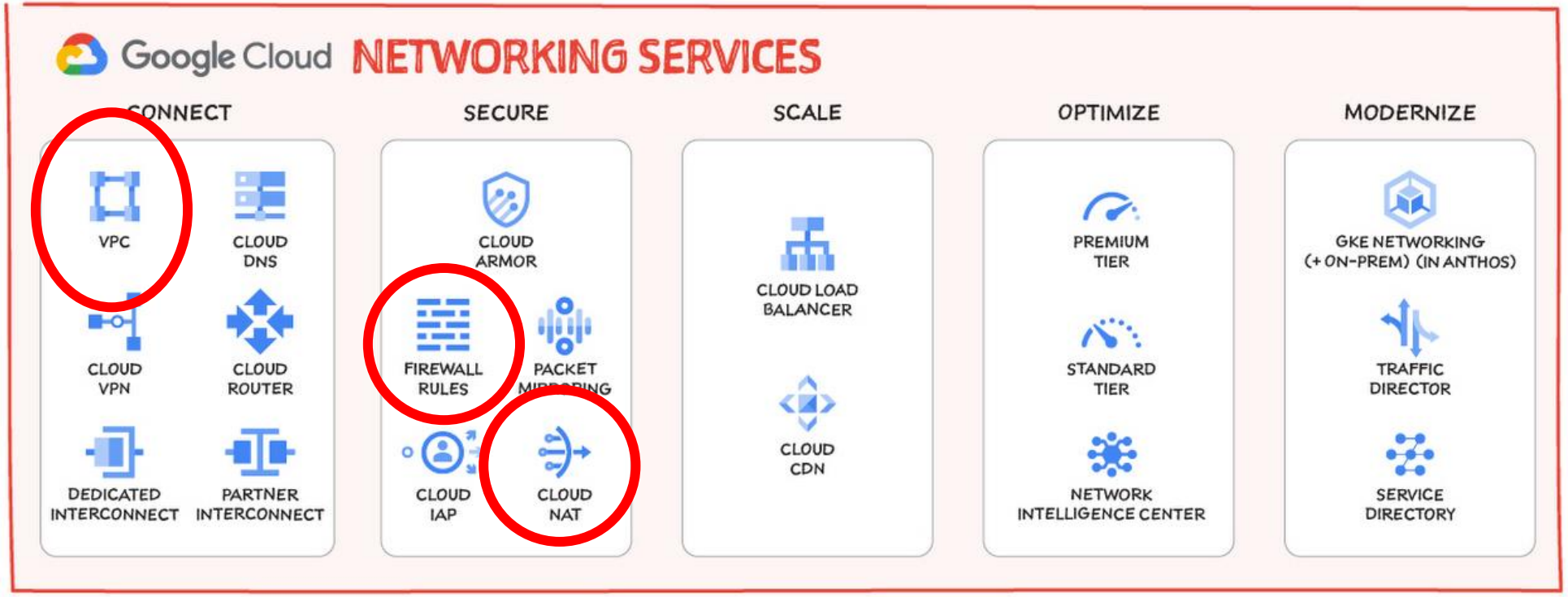
https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-networking-overview

The Physical Infrastructure (WAN and Datacenter)



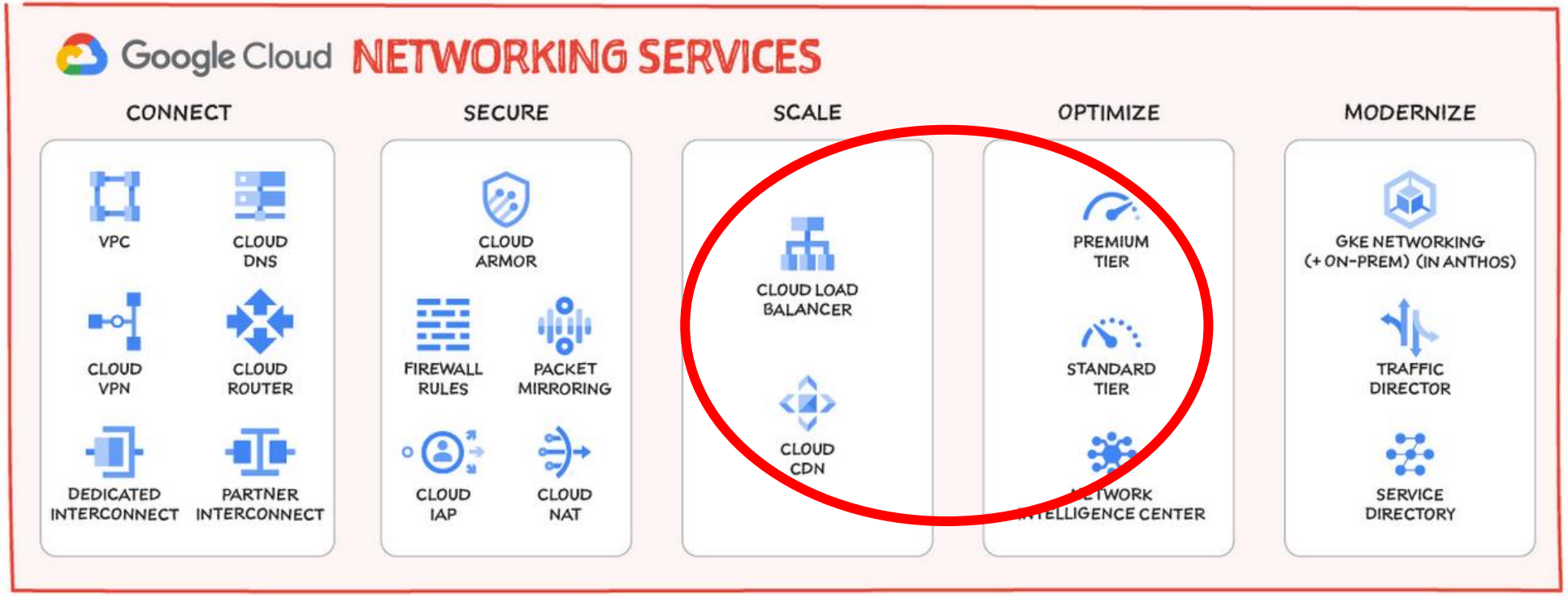
https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-networking-overview

Network Services (previously covered)



https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-networking-overview

Network Services – This Module



https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-networking-overview

Outline of module

- Load Balancing Introduction
- Load Balancing Example
- Autoscaling
- Google’s Maglev
- Content Distribution Networks (CDN)
- Akamai’s Network



University of Colorado **Boulder**

Load Balancing Intro (part 1)

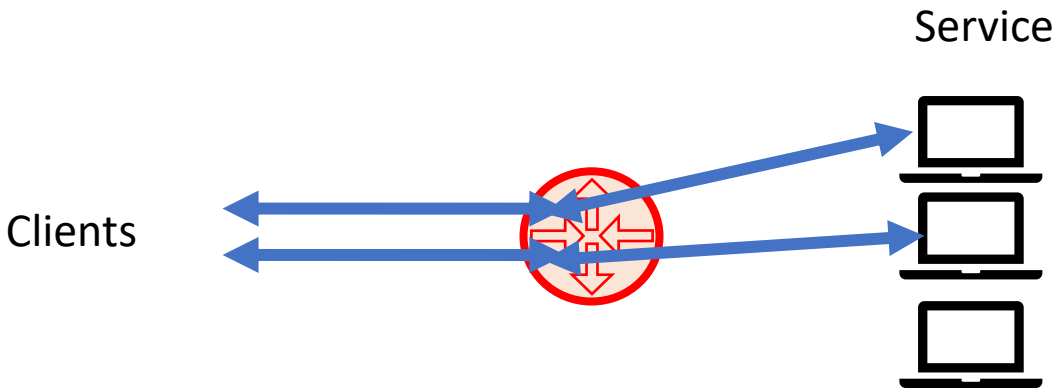
Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

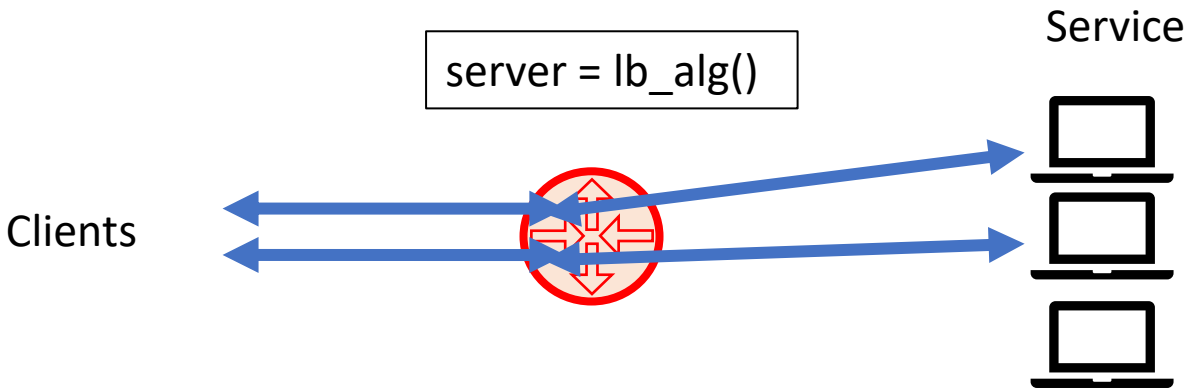
What is Load Balancing

- A load balancer serves as the point of entry for a service, and directs traffic to one of N servers that can handle the request
- Used for both scaling and for resilience



Key Property 1: Algorithm

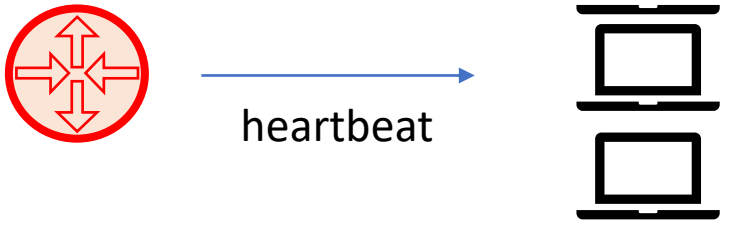
- Client initiates connection
- Load balancer must select which server to forward the request to



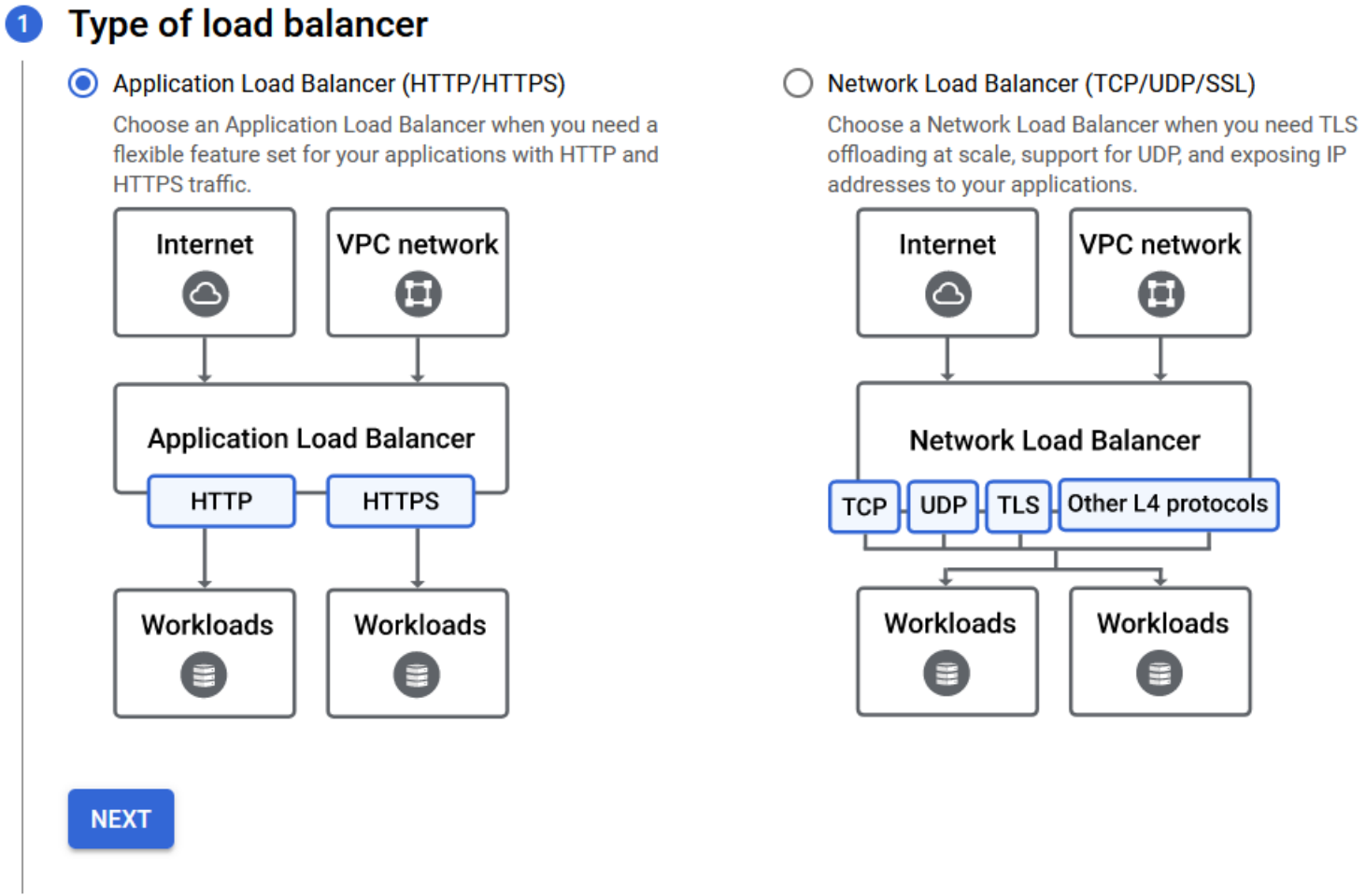
Key Property 2: Server Set

- Which servers are part of the set to choose from
- May include liveness checks (through heartbeats)

Servers	Live
10.0.0.1	Yes
10.0.0.2	Yes
10.0.0.3	No

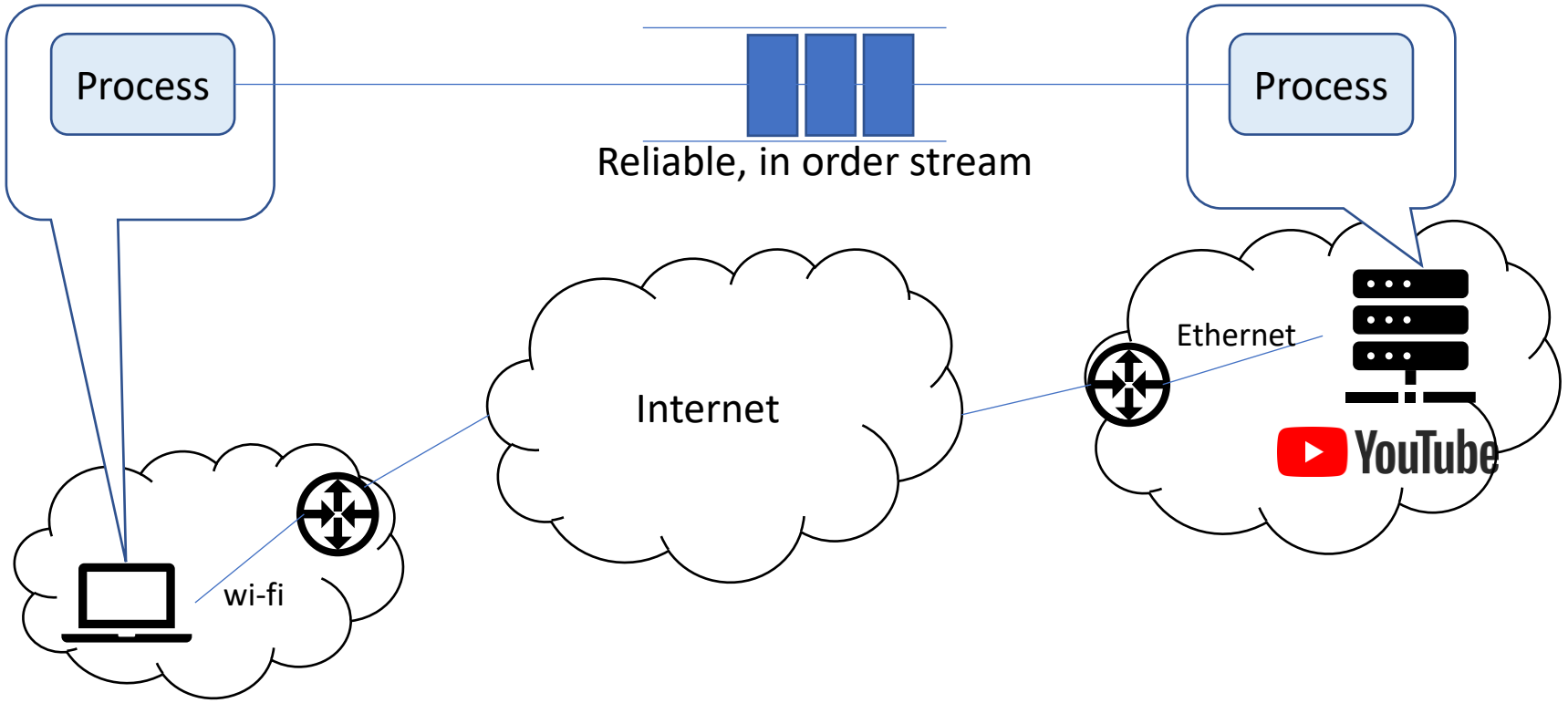


Application Load Balancer (L7) vs Network Load Balancer (L4)

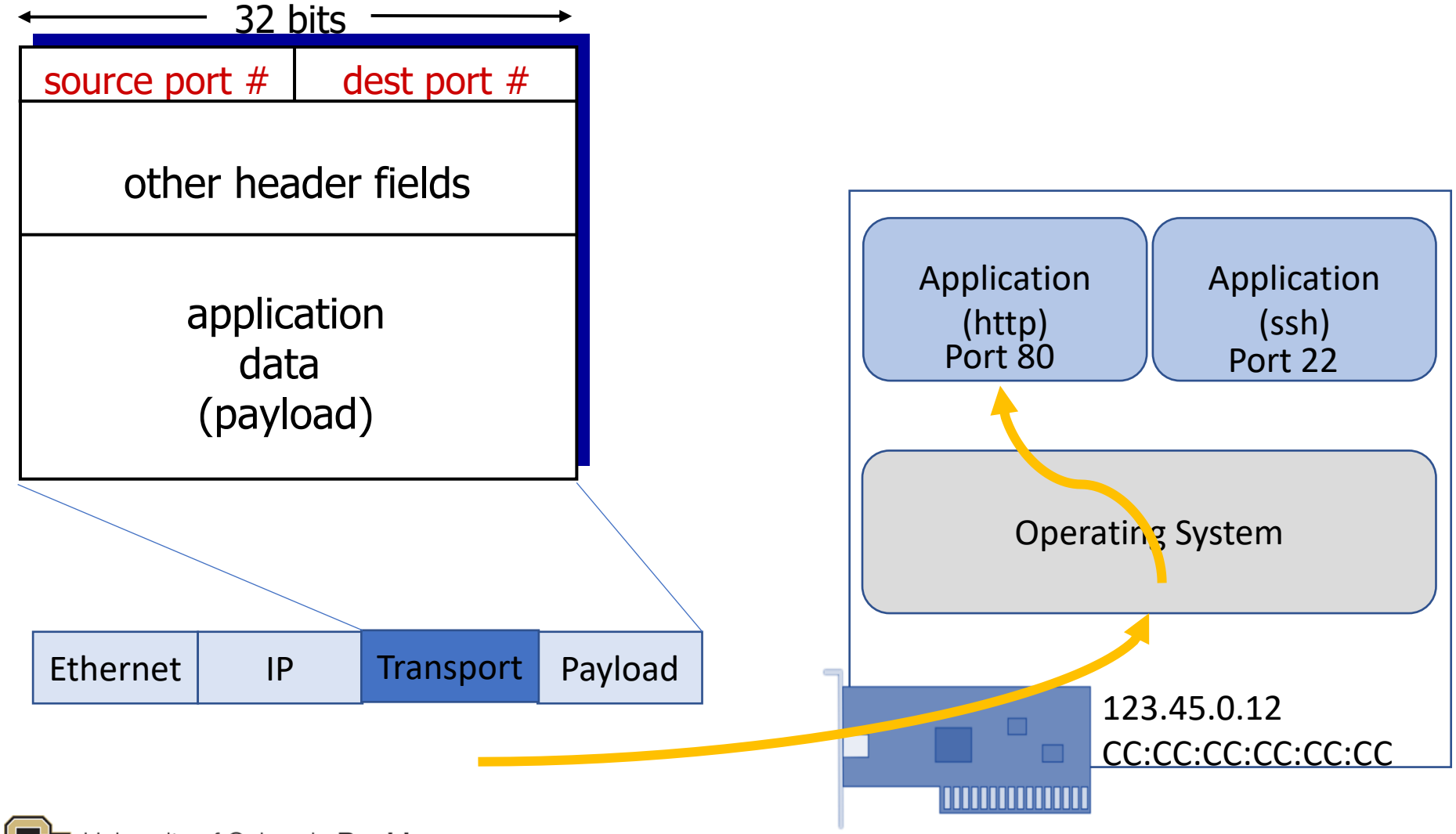


Transport Layer (L4)

- Transport layer enables process-to-process, and reliable in-order stream



Transport Addressing - Ports

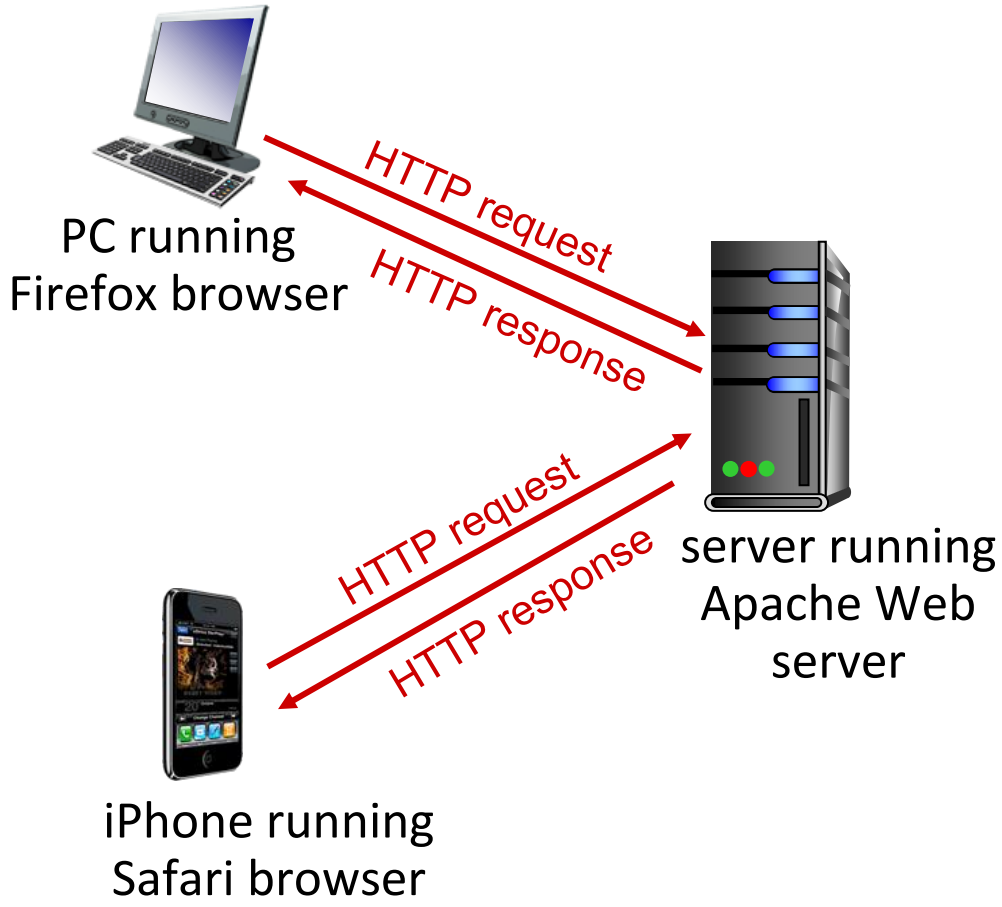


An Application Layer Protocol Defines

- Message syntax – what fields, how fields are delineated
- Types of messages exchanged – e.g., request, response
- Message semantics – meaning of information in fields

Example Application Layer - HTTP

- Web's application-layer protocol
- Client / Server exchange Web Objects
  - Html, jpeg, audio file
- Stateless – server doesn't retain information about past client requests



`http://www.example.com/blah/blah.gif`

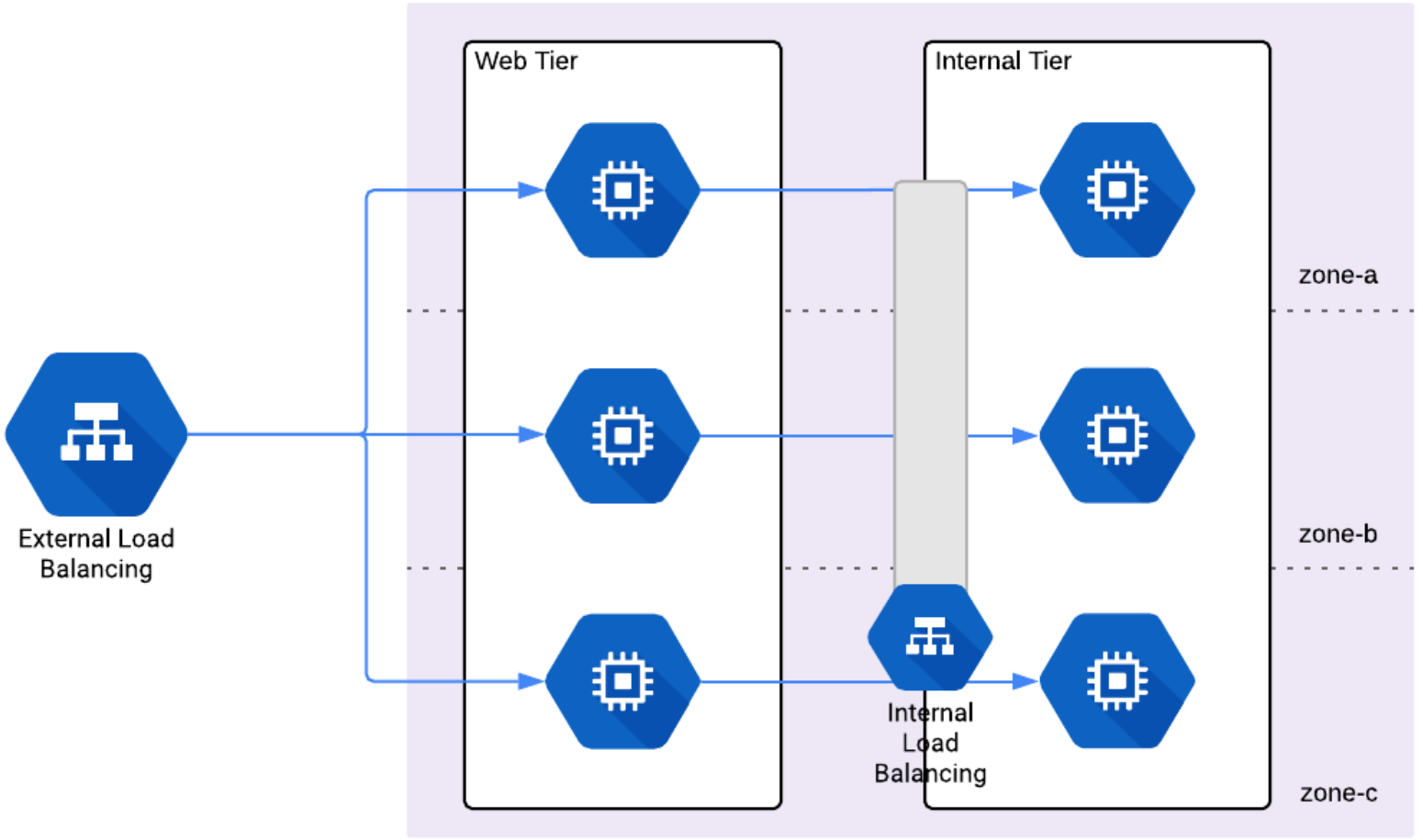
proto host name path

Application Load Balancer (L7) vs Network Load Balancer (L4)

- Layer 4 considers network (IP) and transport (TCP) headers
  - Can passthrough the traffic
  - Routing consideration: how to balance load, and may ensure flow affinity

- Layer 7 also considers application (HTTP) header
  - Also called web proxy
  - Terminates the TCP connection
  - Routing consideration: can also direct specific URLs to specific servers
    e.g., http://<domain>/signup goes to server 1,
    http://<domain>/purchase goes to server 2 or 3

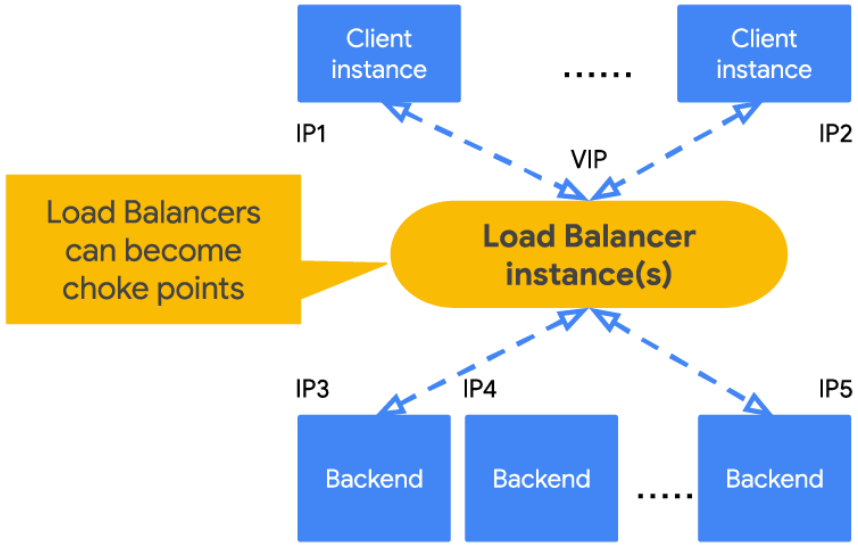
Internal vs External



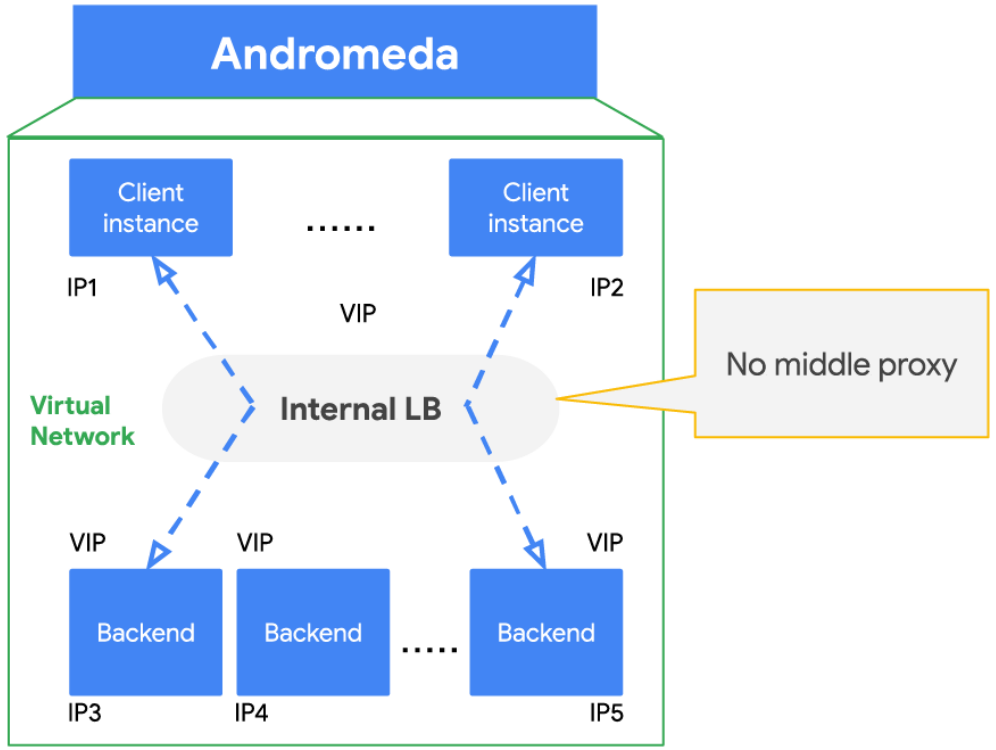
https://github.com/gruntwork-io/terraform-google-load-balancer

Key difference - Andromeda

With Internal Network Load Balancer, forwarding rules can be handled with Andromeda
(Andromeda is the virtual switch running on each host – see previous module)



Typical instance-based Internal Load Balancing



Software-defined Internal Load Balancing on GCP

https://cloud.google.com/blog/products/networking/google-cloud-networking-in-depth-cloud-load-balancing-deconstructed

Other Systems in GCP

- Envoy – for Application Layer Load Balancing (internal or external)
- Maglev – for Network Load Balancer (from external)

Part 1 Summary

- Application (L7) vs Network (L4) Load Balancing
- Internal vs External
- Global vs Regional
- What Premium Tier is and how Global Load Balancing can be realized with Anycast



University of Colorado **Boulder**

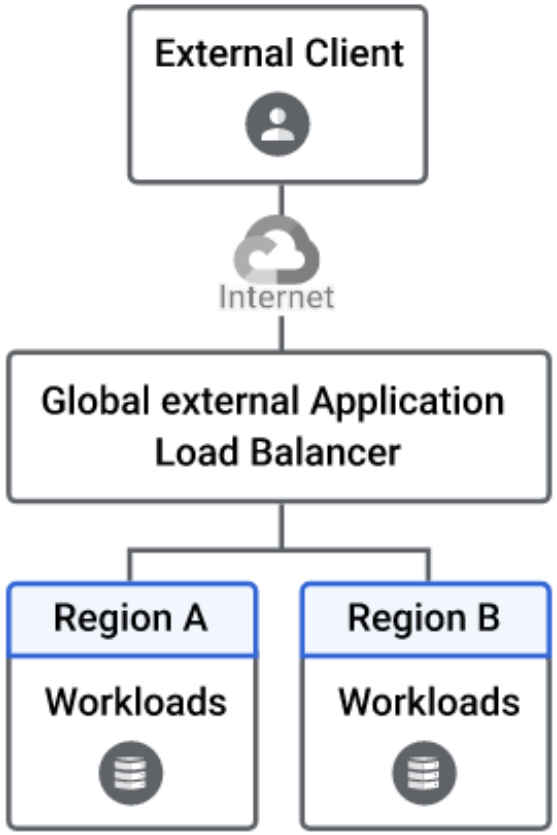
Load Balancing Intro (part 2)

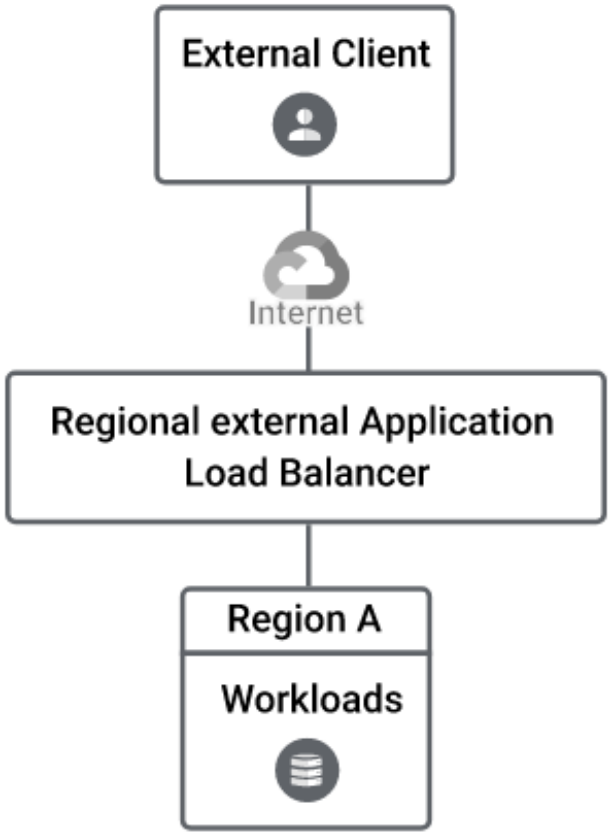
Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

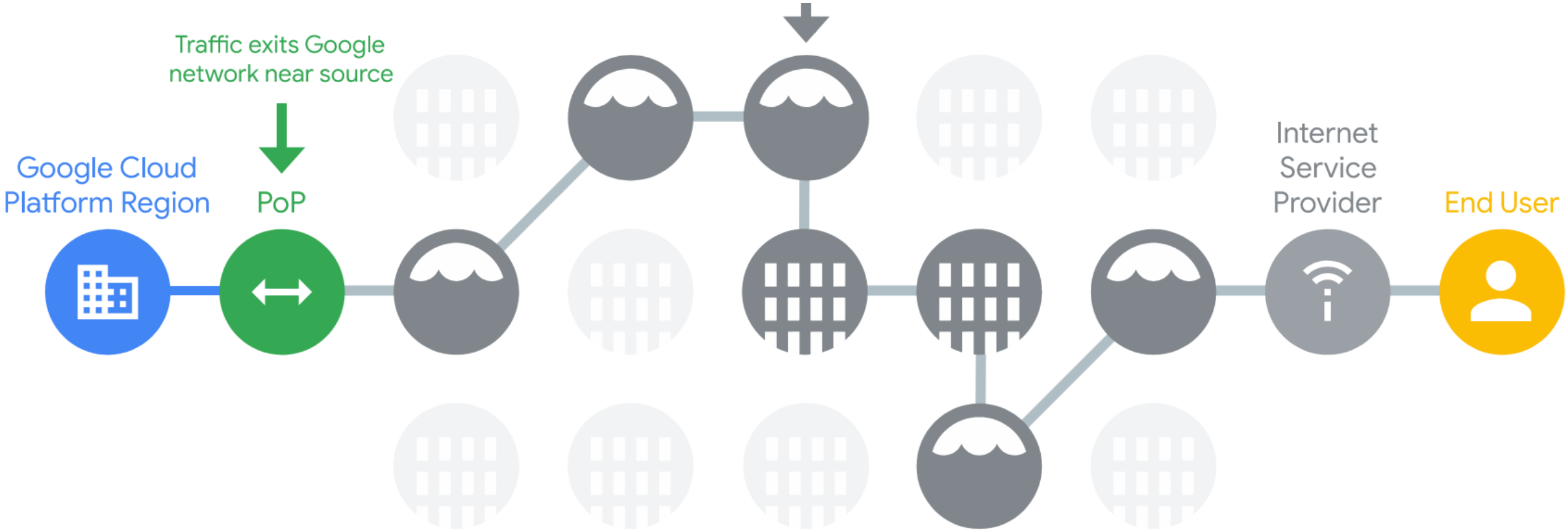
Global vs Regional



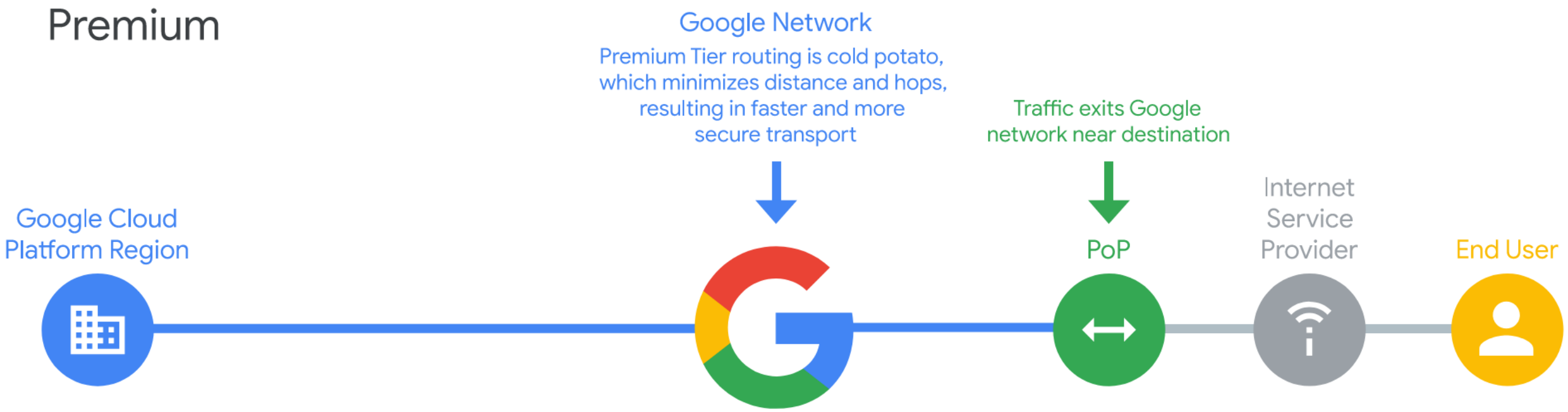


Premium vs Standard Tier

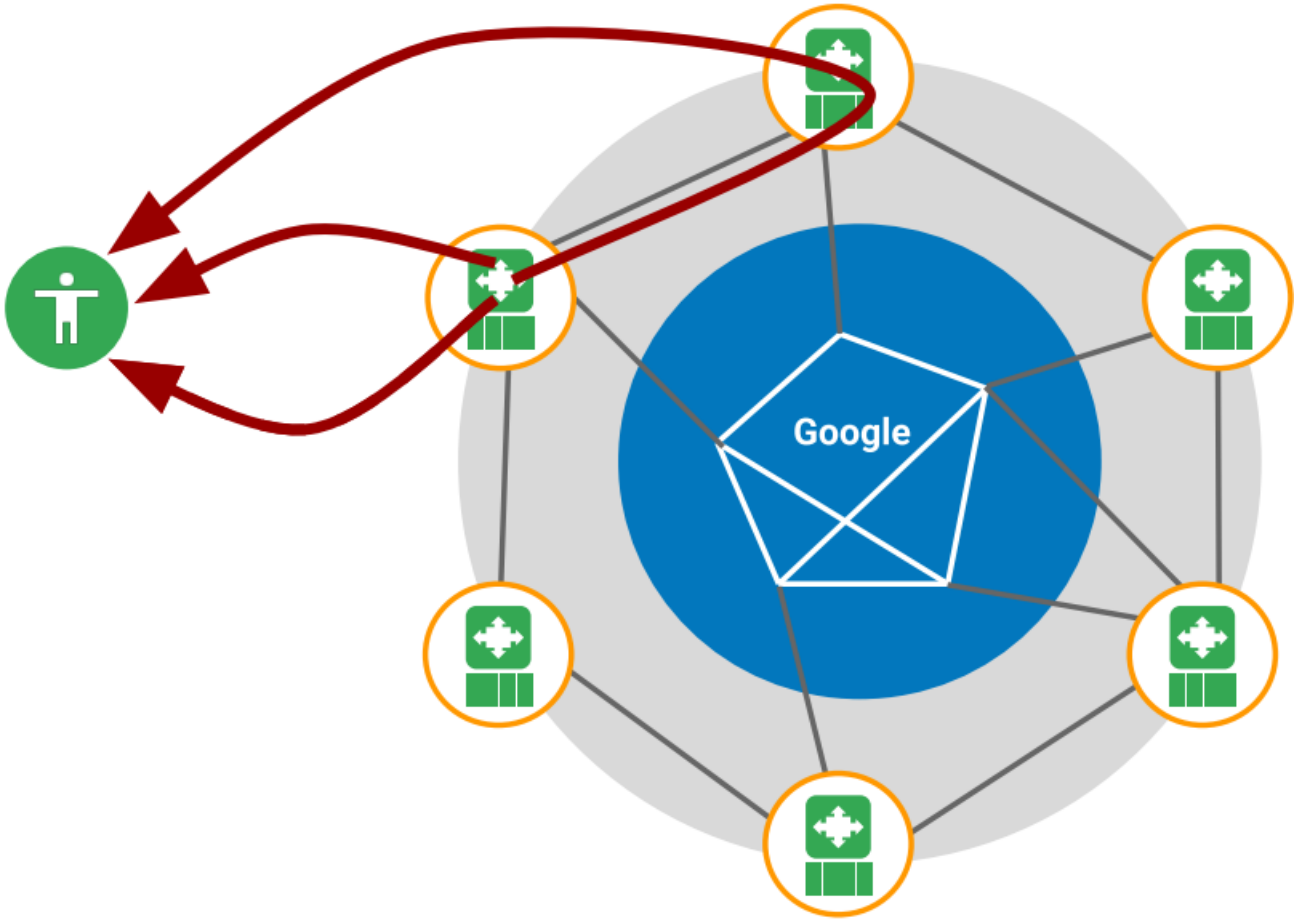
Egress in Standard Tier



Egress in Premium Tier



Recall: Google's Espresso



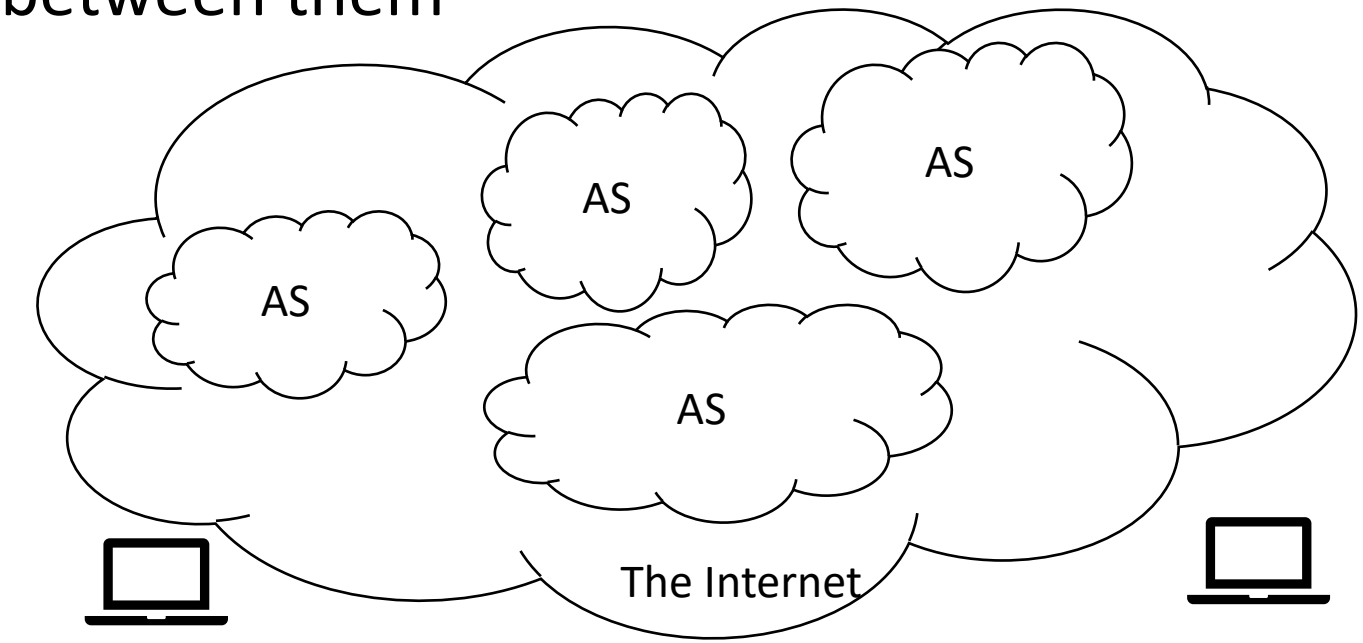
Ingress with Global Load Balancing

- In Premium Tier, the same global external IP address is advertised from various points of presence, and client requests are directed to the client's nearest GFE.

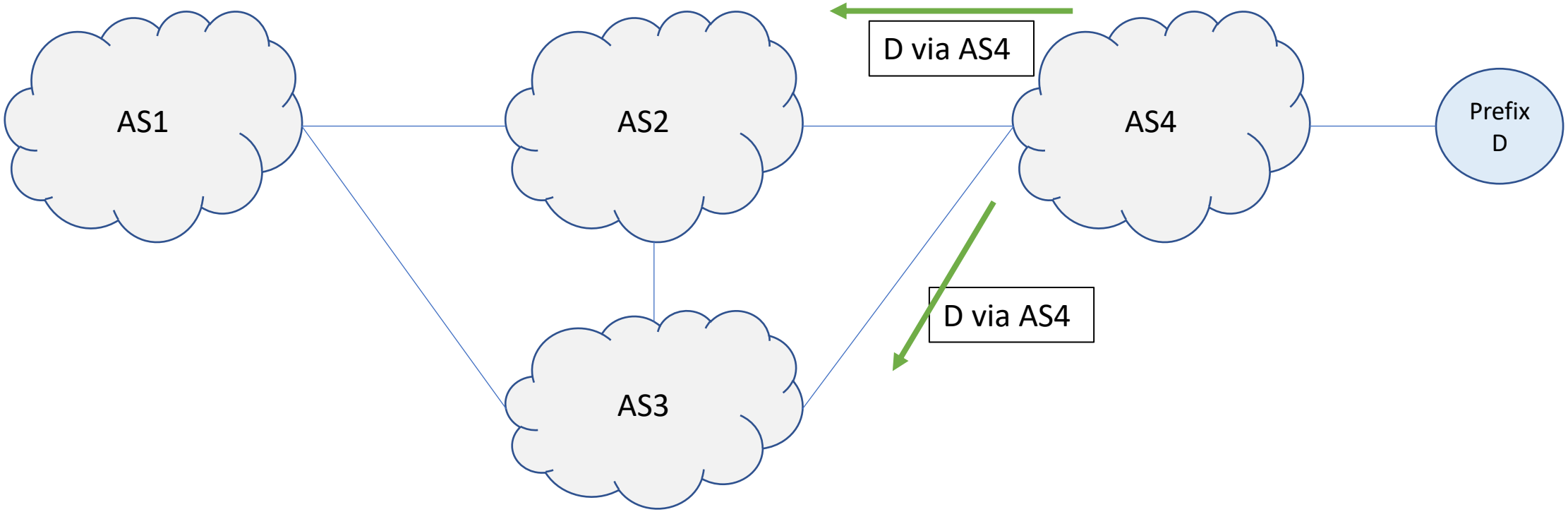
The Internet

Many inter-connected networks called Autonomous Systems

Border Gateway Protocol (BGP) is the protocol used to coordinate between them

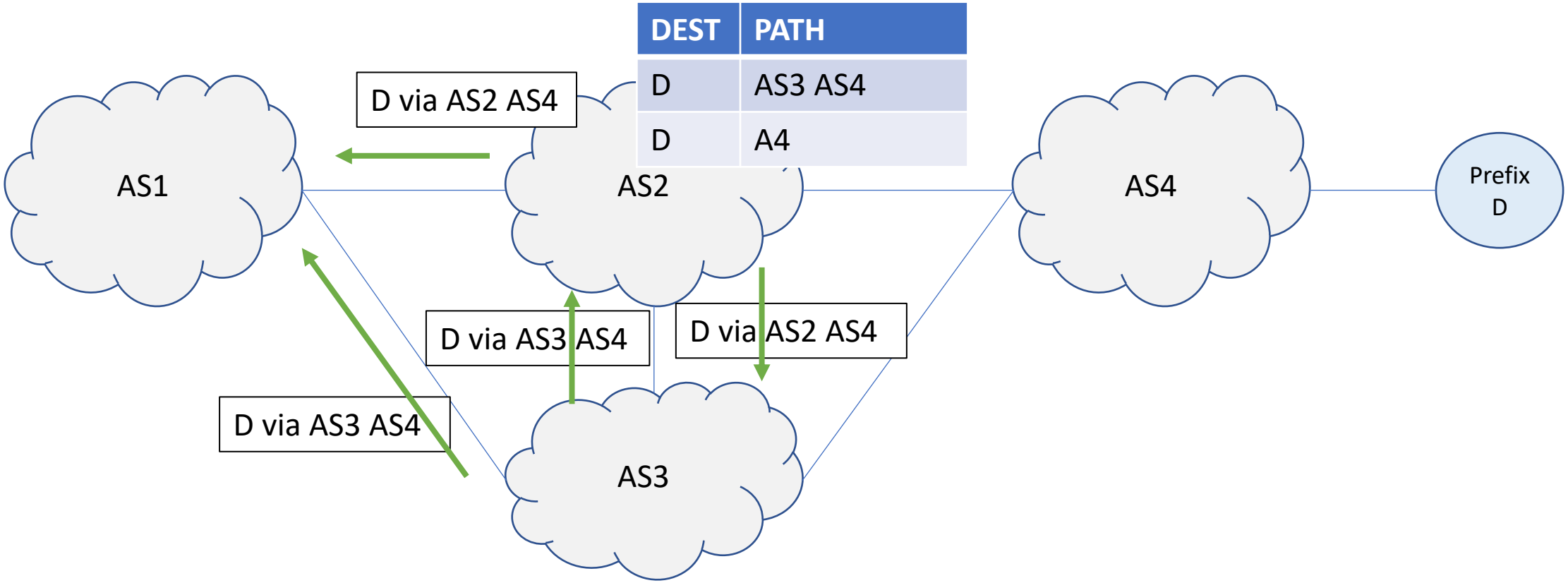


Example of route exchange



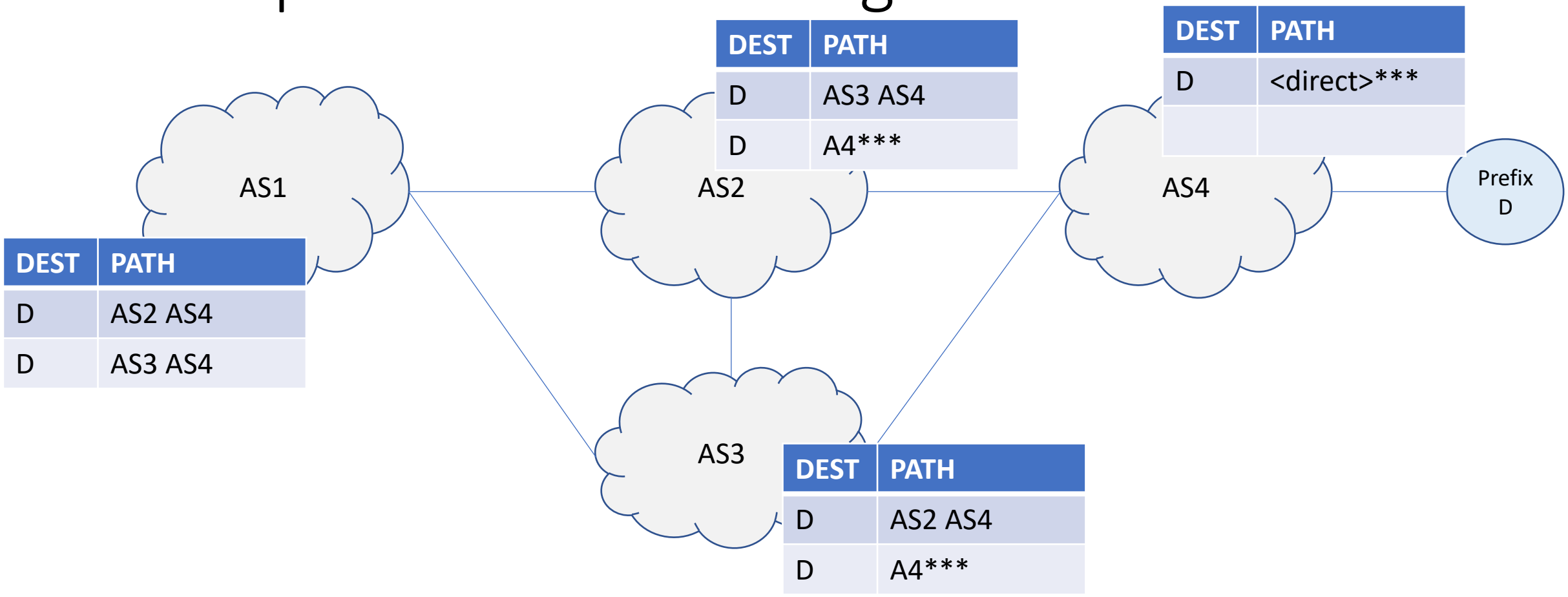
- AS4 announces to its neighbors (AS2 and AS3) that it can reach D with a path of AS4.
- AS2 and AS3 will insert that into the Routing Table and then choose best path to D

Example of route exchange



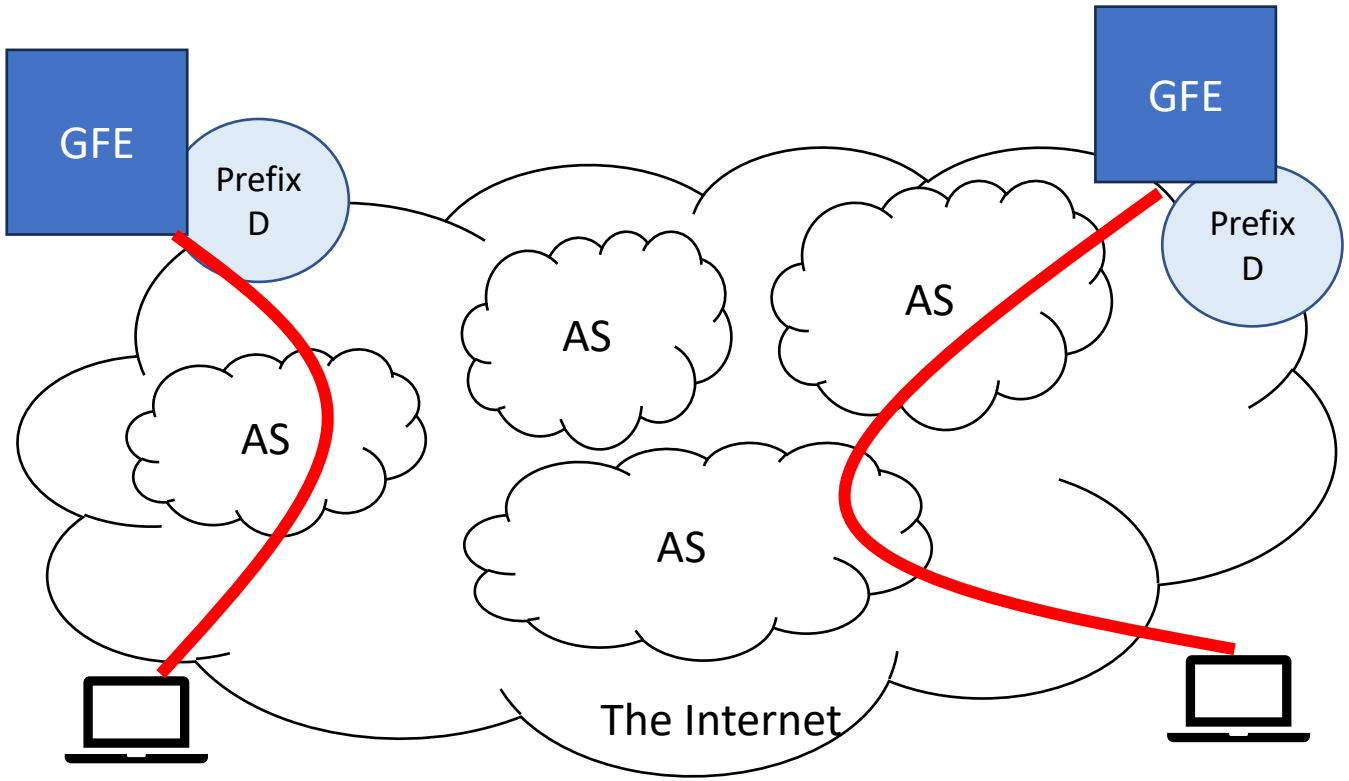
- AS2 and AS3 announce to their neighbors.
  - AS2 and AS3 now each know about 2 paths to D
  - e.g., AS2 knows (i) via AS4, (ii) via AS3 AS4. Will choose (i)

Example of route exchange



- AS1 knows 2 paths (i) via AS2 AS4, (ii) via AS3 AS4. Which will it choose?

Anycast



Summary

- Application (L7) vs Network (L4) Load Balancing
- Internal vs External
- Global vs Regional
- What Premium Tier is and how Global Load Balancing can be realized with Anycast



University of Colorado **Boulder**

Load Balancing Example

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services

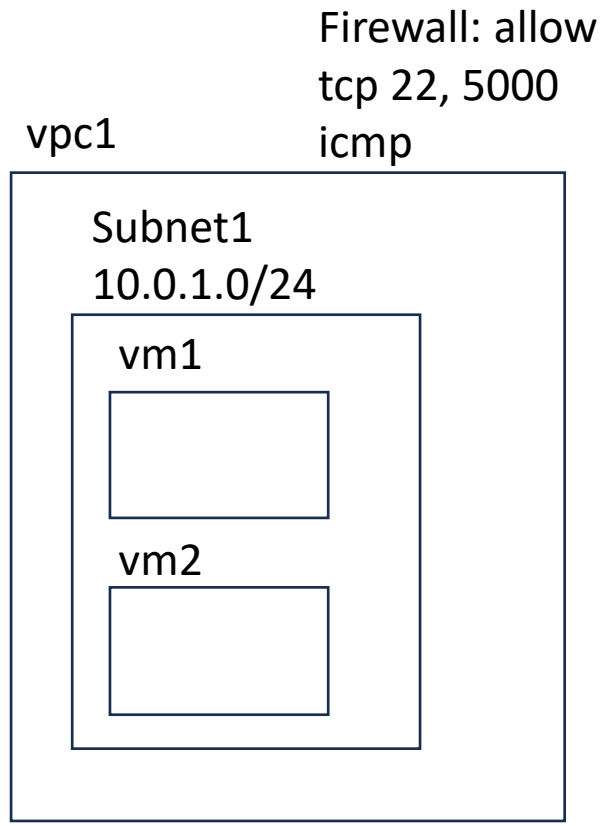


University of Colorado **Boulder**

Setup

Sample Flask App:
https://github.com/eric-keller/npp-cloud/tree/main/test-app

- Create 2 VMs
- ssh into each and run simple Flask application on them
  - sudo apt install python3-flask –y
  - python3 test-app.py
- Create an Unmanaged Instance Group
- Create an Application Load Balancer
  - Frontend
  - Backend
  - Health Check
- curl http://<IP>/hello



Summary of things we saw

- Instance Group
- Proxy Subnet
- Front End
- Backend Service
- Health Check

Terraform

- There’s several components to create, not just one, so this is pointing to examples

https://cloud.google.com/load-balancing/docs/https/ext-http-lb-tf-module-examples



University of Colorado **Boulder**

Autoscaling

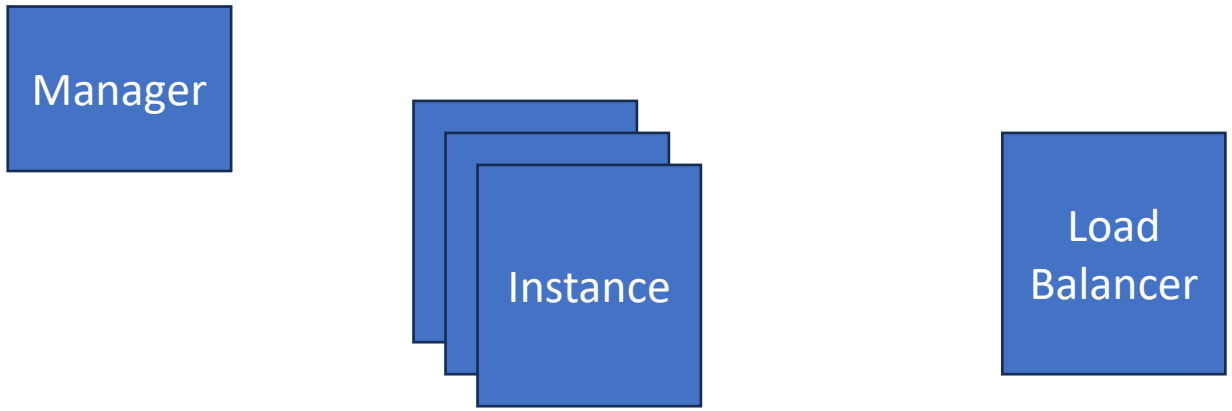
Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

What is Autoscaling

- Monitors usage and creates or kills instances as needed
- Threshold based:
  - E.g., when CPU utilization gets above 50% launch a new VM
- Predictive:
  - Monitor load, use machine learning to predict next load in next [time]
  - See Autopilot – EuroSys 2020



Setup

- Create VM, install software on it
- Create an image
- Use that image in an Instance Template
- Walkthrough autoscaling parameters

Aside on Metadata Server

Aside on Metadata Server

- When a cloud starts, we want to provide configuration information (script, etc)
- This is done by the *metadata service*
  - Similar mechanism in all cloud services
- 169.254.169.254 is an IP address from the reserved IPv4 Link Local Address space 169.254.0.0/16
- Similar to the 10.0.0.0/8 “private addresses”
- But, link local is never routed – it’s local to the current network only

Additional Cloud Services

- Host 169.254.169.254 provides
  - DNS server
  - NTP (time) server
  - Metadata service as a web interface

Accessing metadata

- We can use any web method to access that data
  - Curl, wget
- On boot, startup scripts extract some of the metadata
  - E.g. Startup script

```
curl http://metadata/computeMetadata/v1/instance/attributes/cloud-script -H "Metadata-Flavor: Google" > cloud-script.sh
curl http://metadata/computeMetadata/v1/instance/attributes/service-credentials -H "Metadata-Flavor: Google" > service-credentials.json
curl http://metadata/computeMetadata/v1/instance/attributes/cloud-vm-code -H "Metadata-Flavor: Google" > cloud-vm-code.py
export GOOGLE_CLOUD_PROJECT= $(curl http://metadata/computeMetadata/v1/instance/attributes/project -H "Metadata-Flavor: Google")
```



University of Colorado **Boulder**

Google's Maglev

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

What is Maglev?

- Google’s Network Load Balancer
- Recall: that’s L4 load balancing and is designed to be fast and passthrough traffic

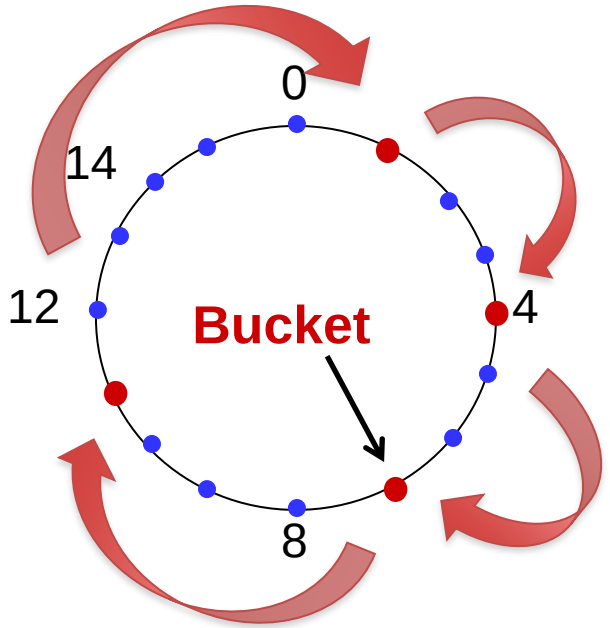
Background on Consistent Hashing

- Originally designed around content distribution (Akamai)
- Suppose we want to partition files (or balance load) among a set of servers
- Could use modulo: server = x modulo k
  - But might not be even distribution
- Use hashing: server = hash(x) modulo k
  - What if k changes (failure or scaling)? Mapping of x to server will change



Consistent Hashing

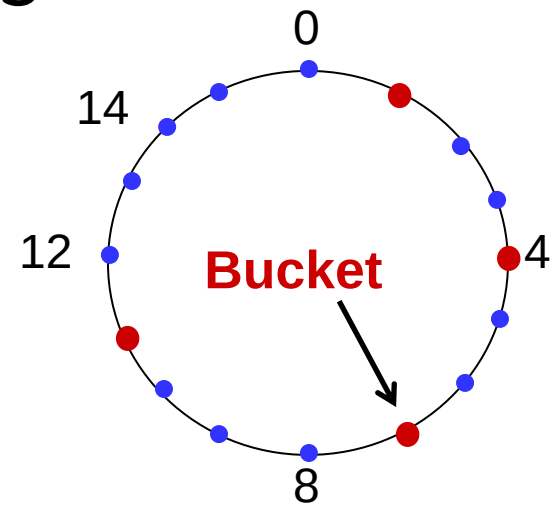
- Construction
  - Assign *n* hash buckets to random points on mod 2^k circle; hash key size = *k*
  - Map object to random position on circle
  - Hash of object = closest clockwise bucket
    - *successor* (key) → bucket



- Desired features
  - Balanced: No bucket has disproportionate number of objects
  - Smoothness: Addition/removal of bucket does not cause movement among existing buckets (only immediate buckets)

Consistent hashing and failures

- Consider network of n nodes
- If each node has 1 bucket
  - Owns 1/nth of keyspace *in expectation*
  - Says nothing of request load per bucket



- If a node fails:
  - Its *successor* takes over bucket
  - Achieves smoothness goal: Only localized shift, not O(n)
  - But now successor owns 2 buckets: keyspace of size *2/n*
- Instead, if each node maintains *v* random nodeIDs, not 1
  - "Virtual" nodes spread over ID space, each of size *1 / vn*
  - Upon failure, v successors take over, each now stores *(v+1) / vn*

Maglev

- Danielle E. Eisenbud, et. al. Maglev: A Fast and Reliable Software Network Load Balancer. In Usenix Network Systems Design and Implementation (NSDI), 2016.

- [Switch to their slides]



University of Colorado **Boulder**

Content Distribution Networks (CDN)

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services

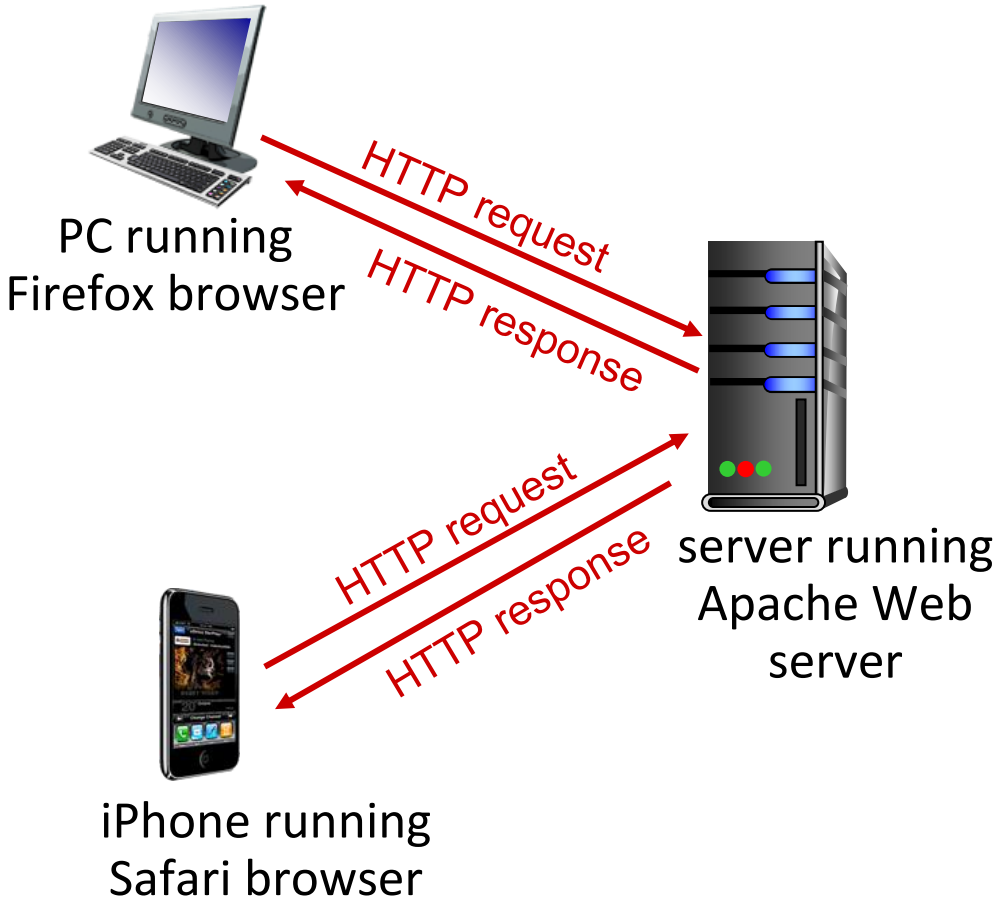


University of Colorado **Boulder**

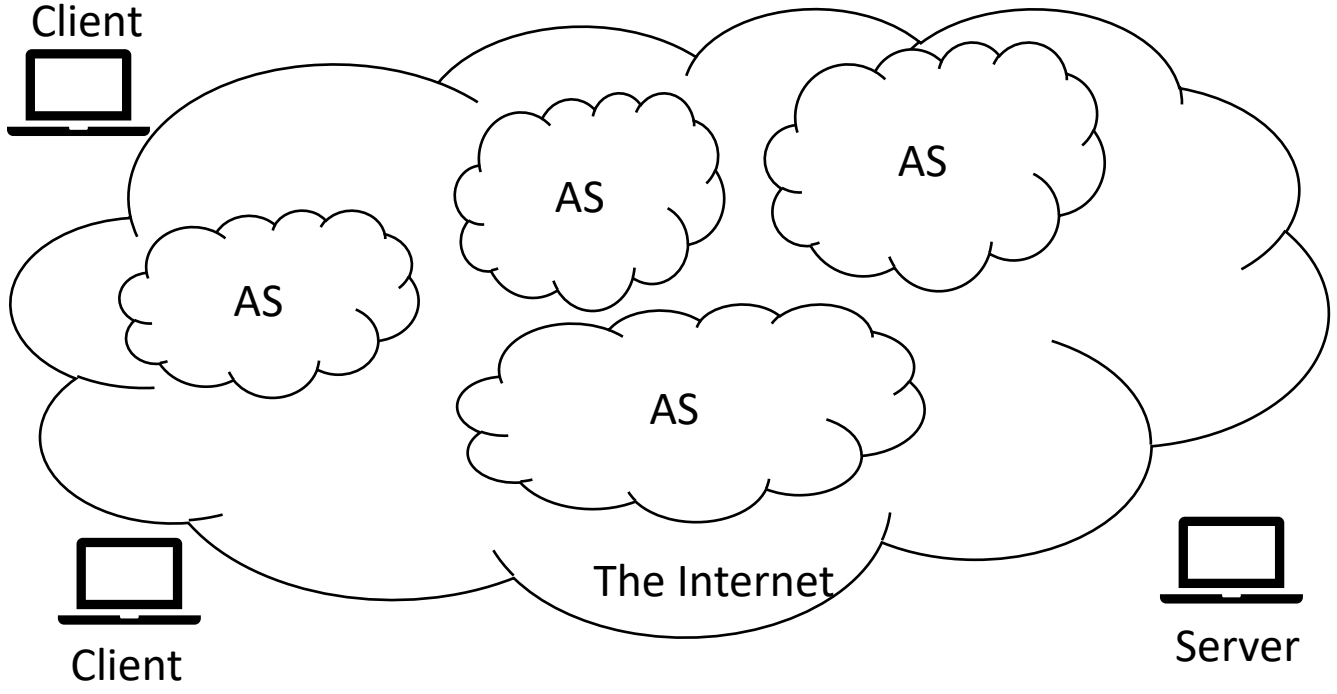
HTTP Review

- Web's application-layer protocol
- Client / Server exchange Web Objects
  - Html, jpeg, audio file
- Stateless – server doesn't retain information about past client requests

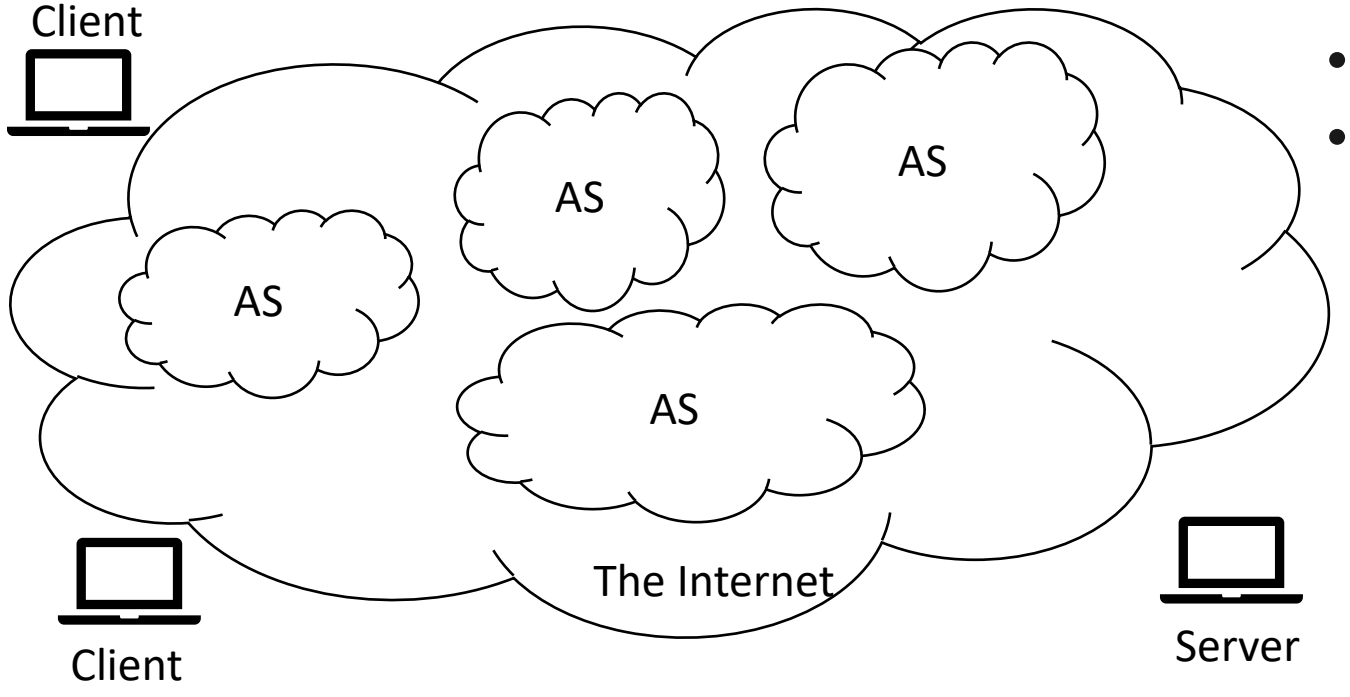




Request over the Internet



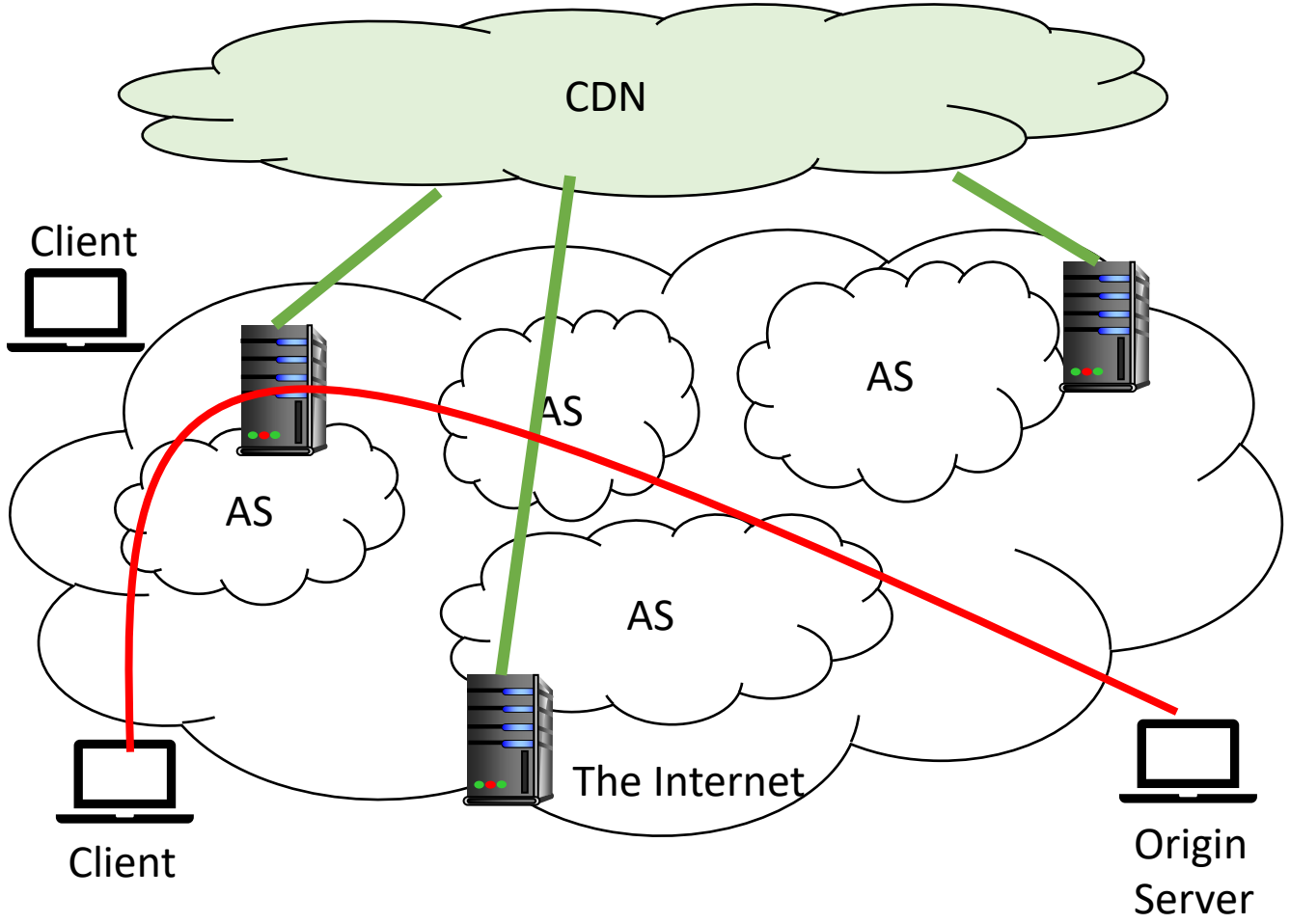
Request over the Internet (to single server)



Poor performance/reliability/scalability:

- Single point of failure
- Easy to be overloaded
- Long latency
- A lot of peering traffic

Content Distribution Network (CDN)



- CDN provider creates network with servers at edge and a lot of peering agreements with network providers
- Content provider signs up with CDN provider and specifies Origin Server
- Client requests go to CDN servers
- First access: CDN Server contacts origin server (then caches content)
- Subsequent accesses: serve cached

GCP CDN Walkthrough

- Create Origin
- Point it to a Backend Service
- Configure how it should cache content



University of Colorado **Boulder**

Akamai's Network

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

Akamai’s Network

- Erik Nygren, Ramesh K. Sitaraman, and Jennifer Sun. 2010. The Akamai network: a platform for high-performance internet applications. SIGOPS Oper. Syst. Rev. 44, 3 (July 2010), 2–19.

Internet Application Requirements

- Application performance is directly tied to key business metrics such as application adoption and site conversion rates.
- Experiment - site visitors were partitioned:
  - half were directed to the site through Akamai (CDN)
  - other half were sent directly to the site's origin servers.
- Finding: through Akamai users were 15% more likely to complete a purchase and 9% less likely to abandon the site after one page
- Another study: Annual revenue increases of $200,000 to over $3 million directly attributable to the improved performance and reliability of their applications

The problem with the Internet

- Reach of any single network is limited
- Peering point congestion
- Inefficient routing protocols
- Unreliable networks
- Inefficient communication protocols
- Scalability (of backend)
- Application limitations and slow rate of change adoption (supporting different browsers)

Table 1: Effect of Distance on Throughput and Download Time

Distance (Server to User)	Network RTT	Typical Packet Loss	Throughput	4GB DVD Download Time
Local: <100 mi.	1.6 ms	0.6%	44 Mbps (high quality HDTV)	12 min.
Regional: 500–1,000 mi.	16 ms	0.7%	4 Mbps (basic HDTV)	2.2 hrs.
Cross-continent: ~3,000 mi.	48 ms	1.0%	1 Mbps (SD TV)	8.2 hrs.
Multi-continent: ~6,000 mi.	96 ms	1.4%	0.4 Mbps (poor)	20 hrs

Akamai's network

- 61,000 servers located across
- nearly 1,000 networks in
- 70 countries worldwide

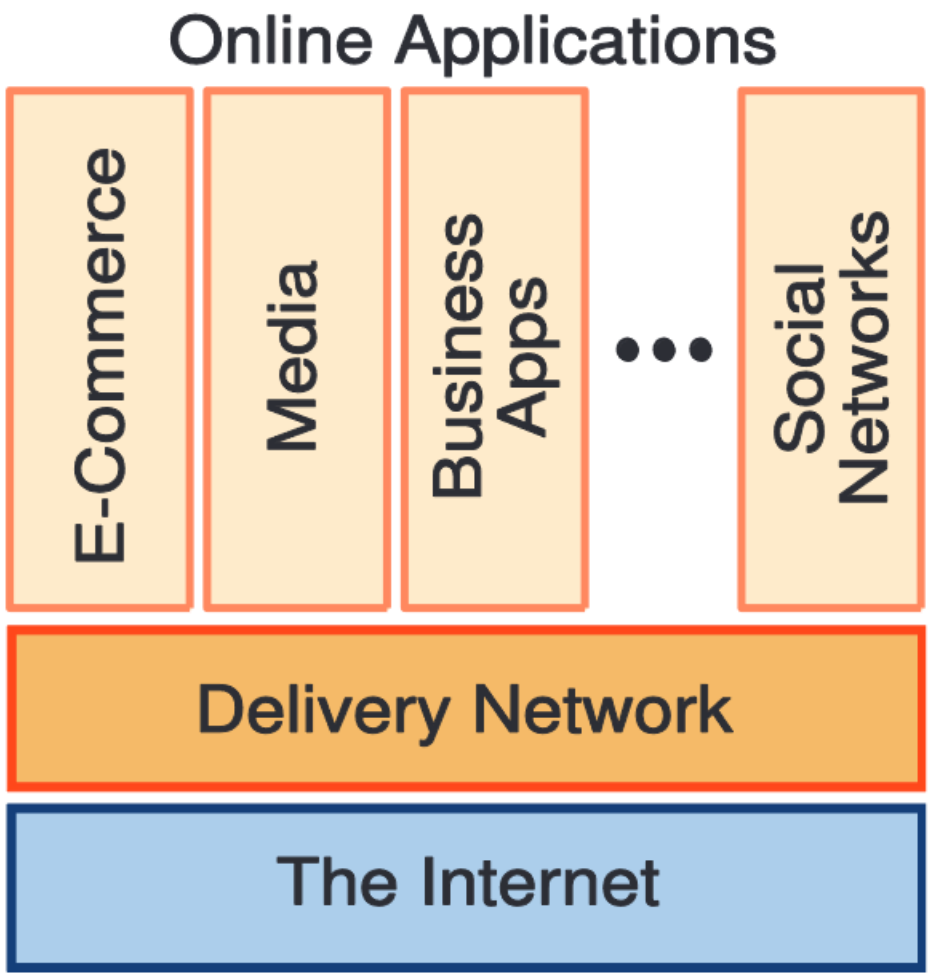
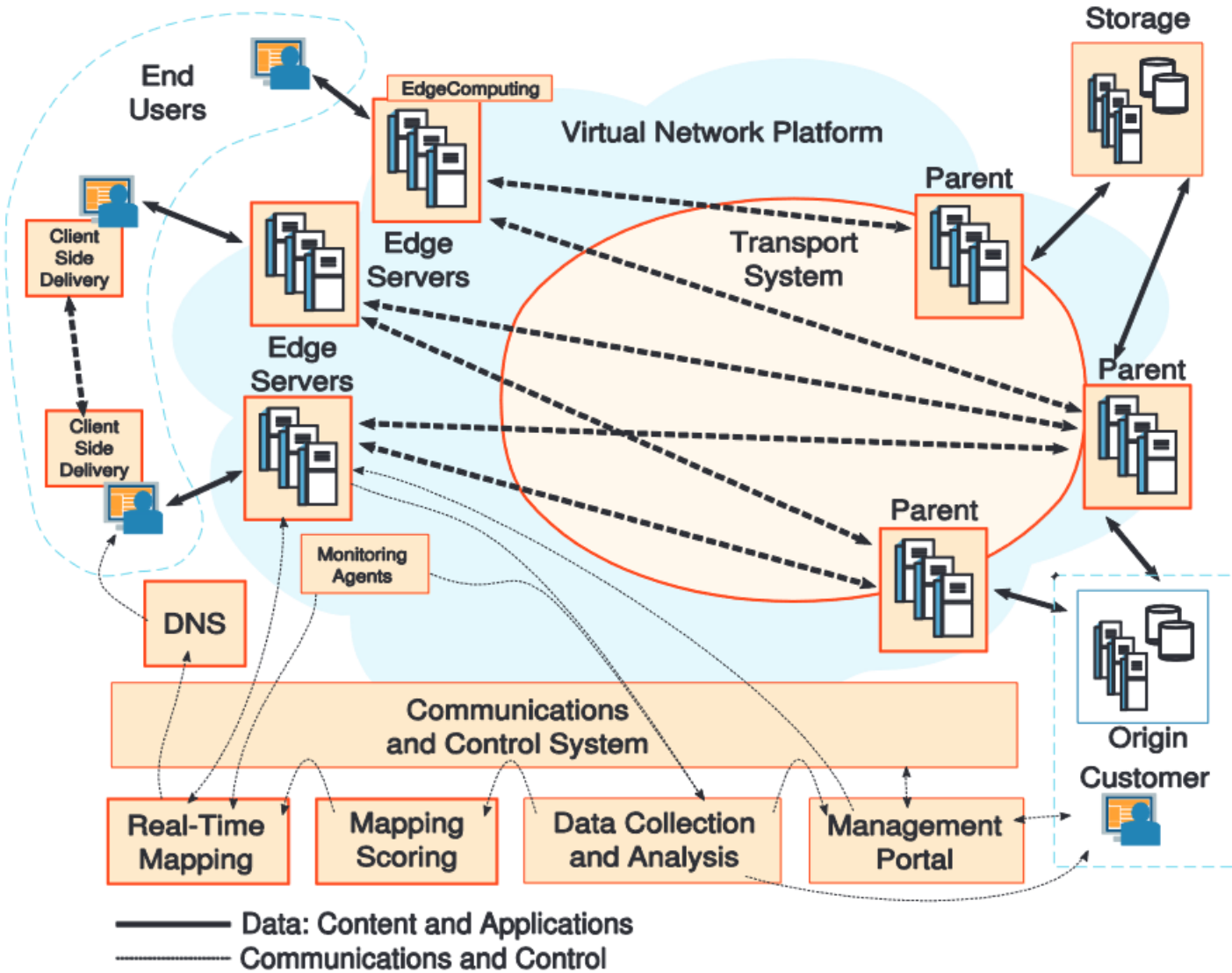


Figure 2: A delivery network is a virtual network built as a software layer over the Internet that is deployed on widely distributed hardware.

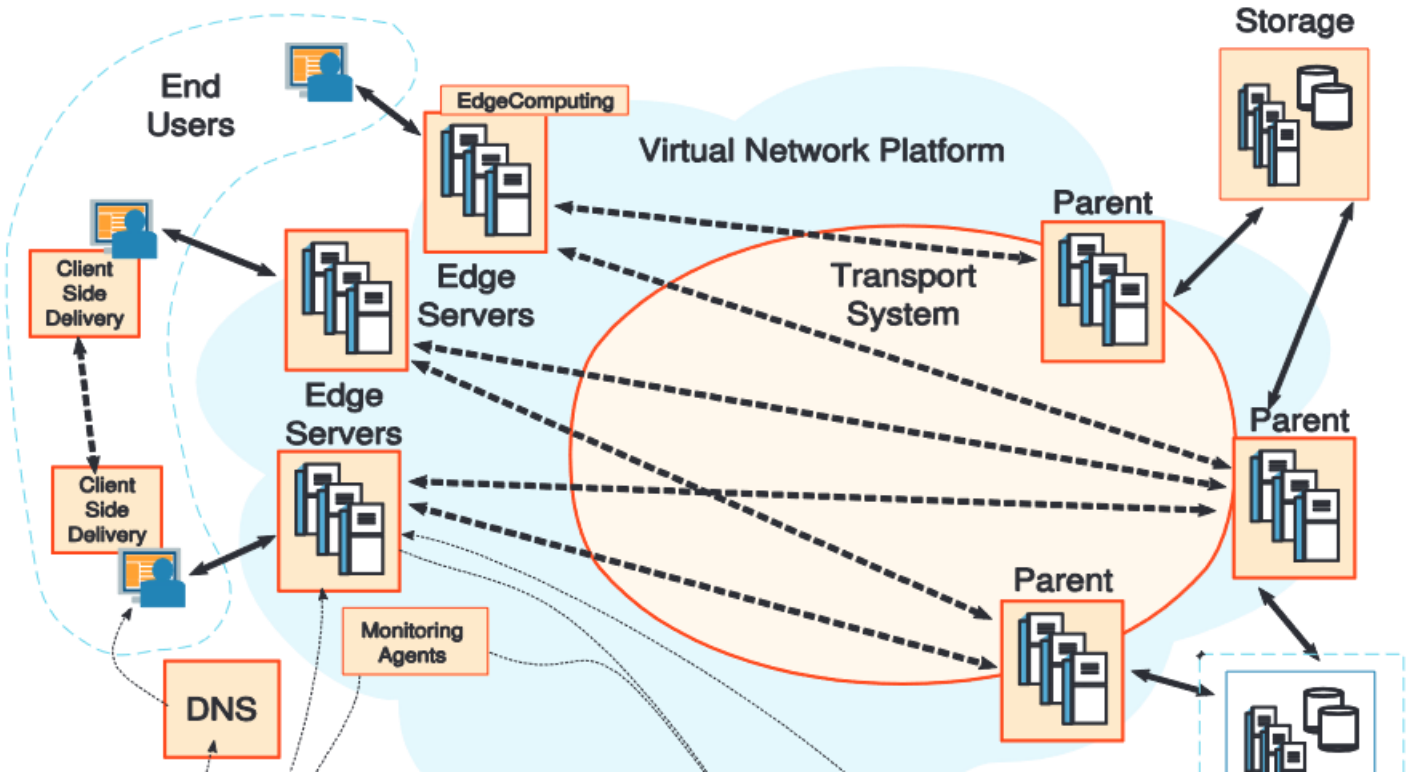

End Users
EdgeComputing
Virtual Network Platform
Storage
Parent
Client Side Delivery
Edge Servers
Transport System
Edge Servers
Parent
Client Side Delivery
Parent
Monitoring Agents
DNS
Communications and Control System
Origin
Customer
Real-Time Mapping
Mapping Scoring
Data Collection and Analysis
Management Portal
Data: Content and Applications
Communications and Control

Use case 1: Content Delivery



Figure 4: With real-time path optimization, Akamai helps customers avoid connectivity problems such as those depicted here, arising from the Middle East cable cuts.

Use case 2: Live video streaming

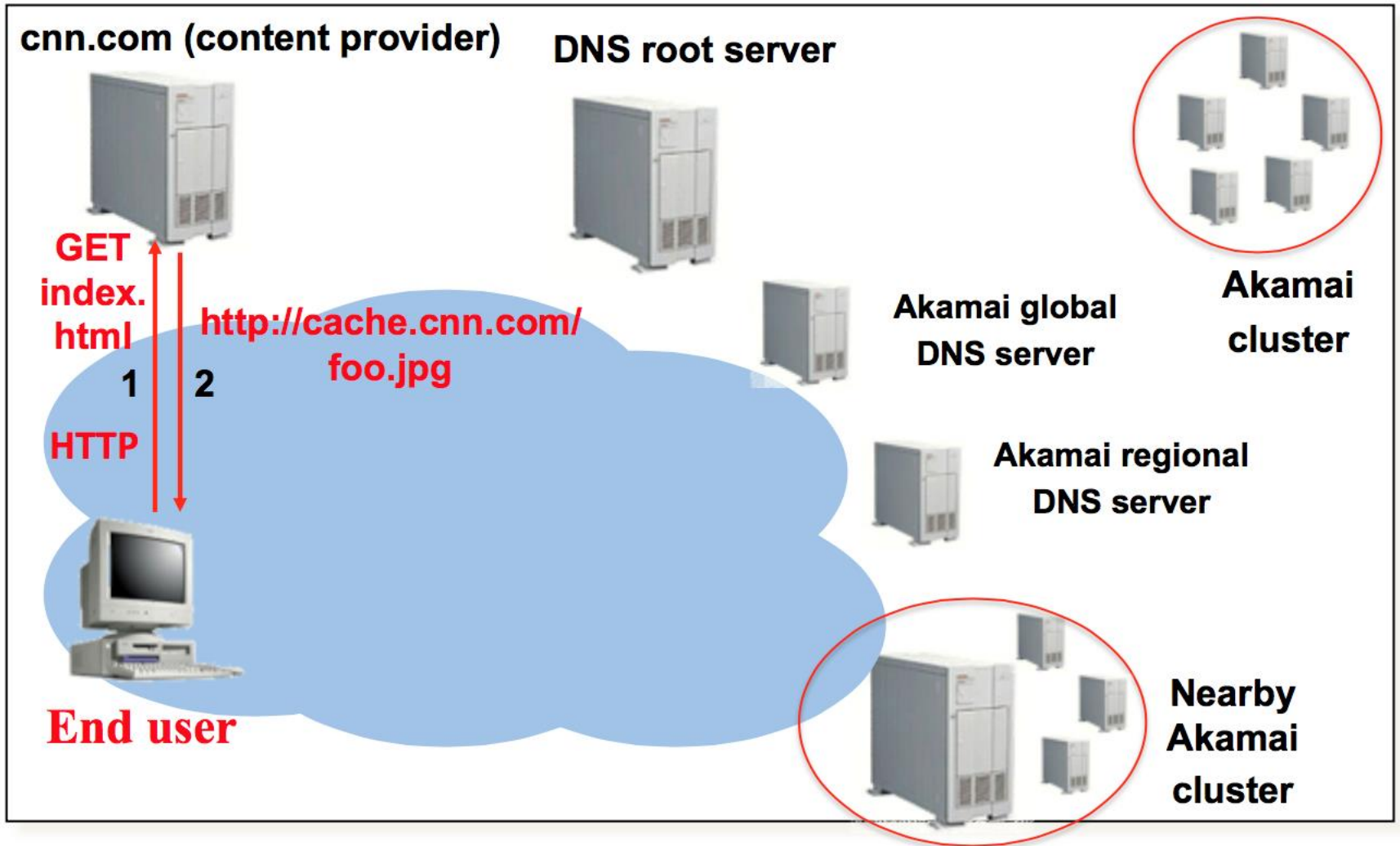


Use case 3: Application delivery

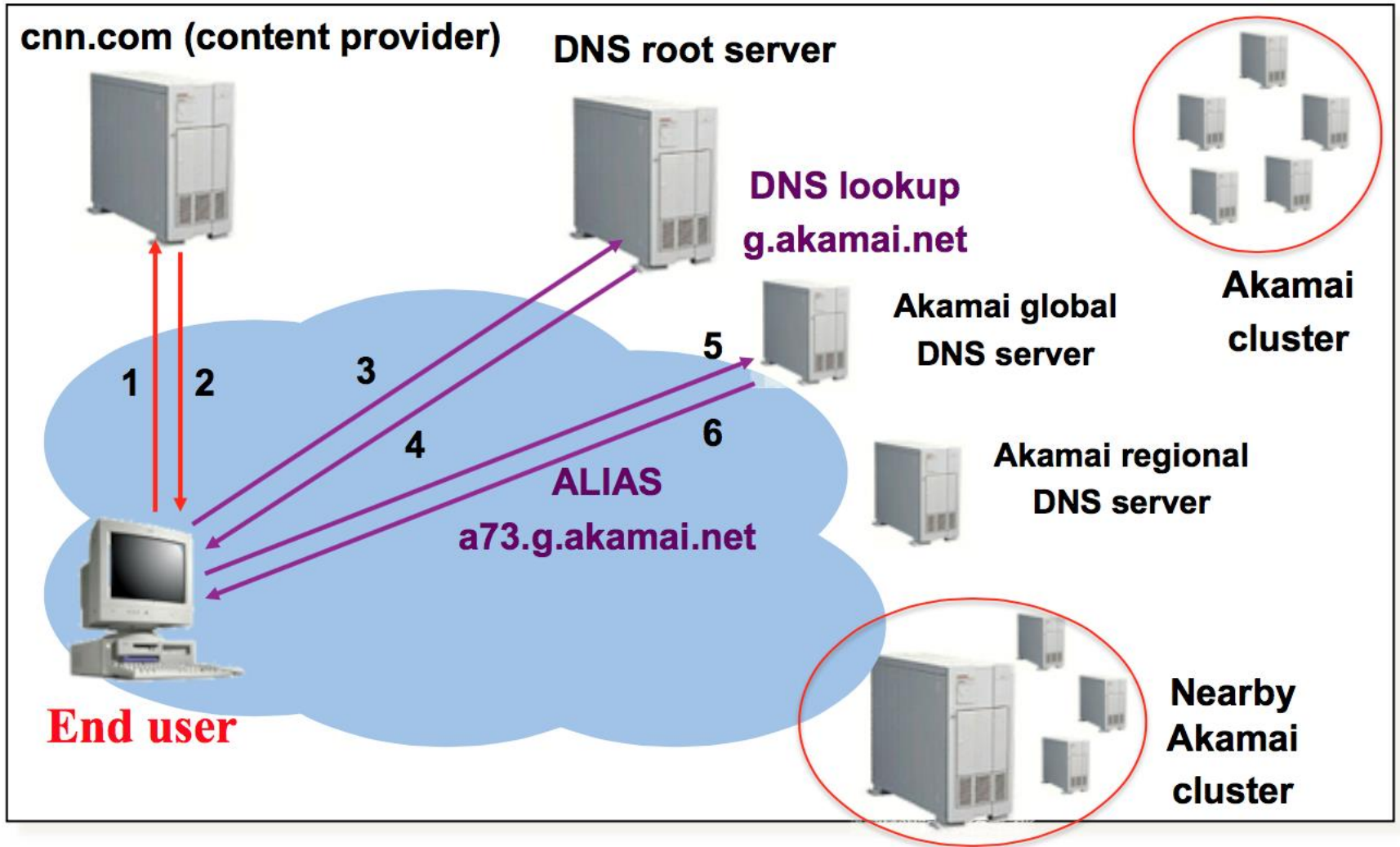
When content is not cachable, still has benefits

- Path optimization (resilient overlay networks – SOSP 2001)
- Packet loss reduction (with forward error correction)
- Transport protocol optimizations (e.g., pooling persistent connections, retransmission based on network observation)
- Application optimization (parse and prefetch content)

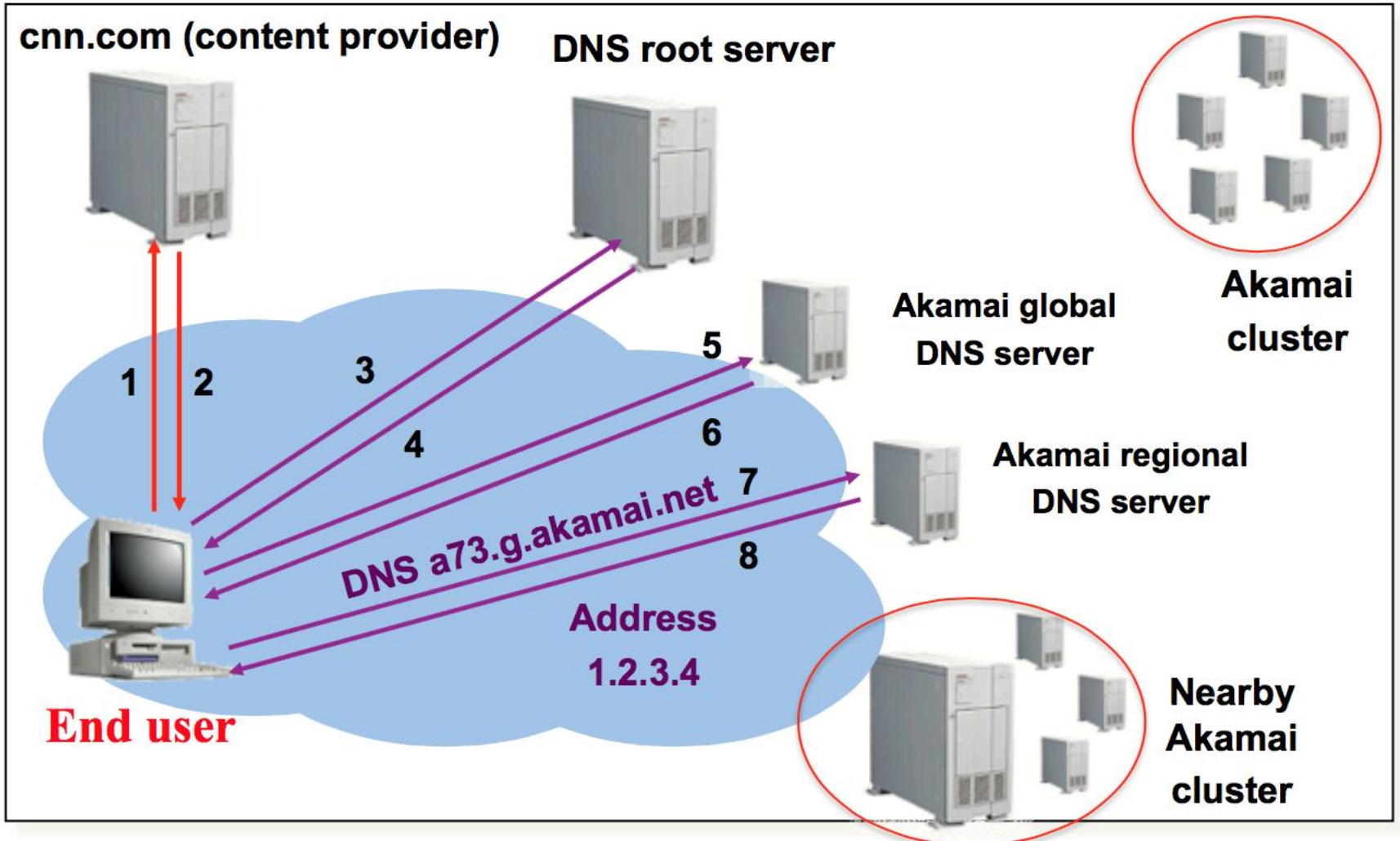
DNS Based Load Balancing



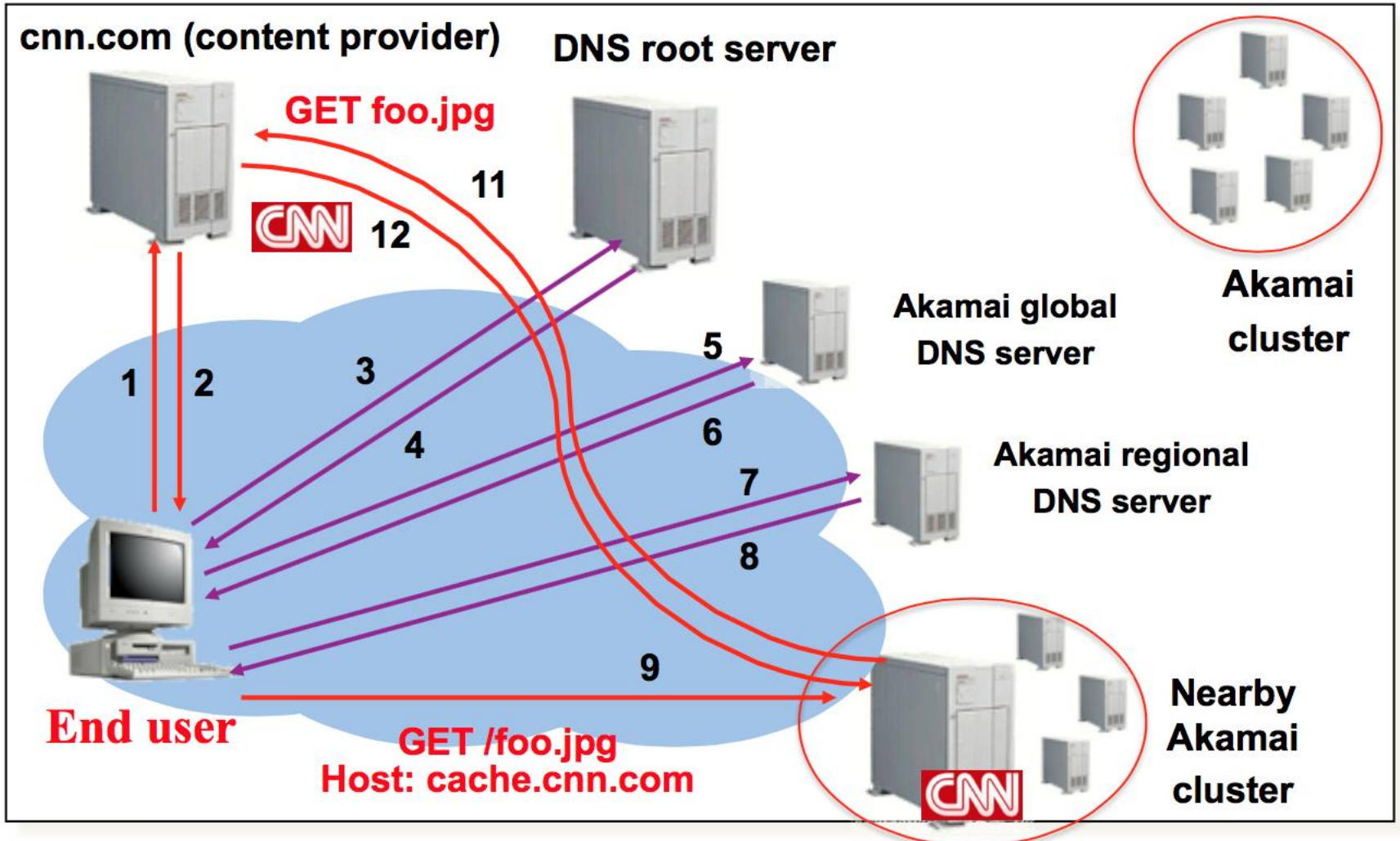
Content is addressed with Akamai.net



Akamai uses Geo IP lookup in DNS servers



Returned IP will be near end user



A lot more in the paper

- How they do analysis
- How they handle failure
- And lots of data on end user benefits



University of Colorado **Boulder**

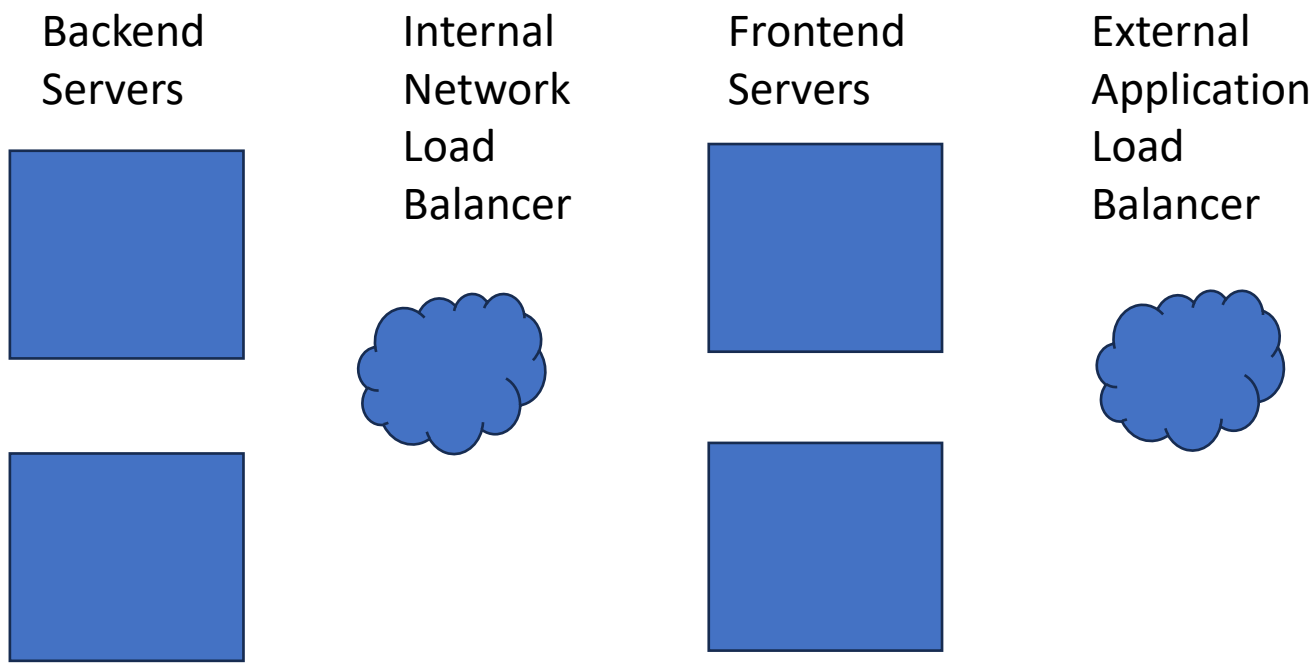
Lab Overview

Course: Networking Principles in Practice – Cloud Networking
Module: Network Services



University of Colorado **Boulder**

Create a Sample Web Service (suggested lab – ungraded)



- Start with Unmanaged, Regional
- Expand to autoscaled (managed) frontend
- Expand to global load balancing (across 2 regions)

Backend can be simple Flask app (return hostname)

Front can be simple Flask app that calls the backend and returns hostname + backend returned value



University of Colorado **Boulder**